

คู่มือการใช้งาน CUAIM Cluster

สารบัญ

คู่มือการใช้งาน CUAIM Cluster	1
สารบัญ	2
1. รายละเอียด IP Address	4
2. รายละเอียดพื้นที่ใช้งานใน CUAIM Cluster	4
3. รายละเอียด User	6
4. การใช้งาน CUAIM Cluster ผ่านโปรแกรม SSH	7
5. การใช้งาน Slurm Workload Manager	8
1) การใช้งาน Slurm	8
2) การตรวจสอบสถานะต่าง ๆ ภายในระบบ	8
3) การตรวจสอบสถานะงานภายในคิว	11
4) การตรวจสอบทรัพยากรที่ใช้งานได้ทั้งหมด	12
5) การตรวจสอบ QOS ในระบบ	13
6) การ Submit Job	15
6.1) Batch job	16
6.2) Interactive job	17
6.2) Submit job with QOS	17
7) การตรวจสอบสถานะของงาน	18
8) การยกเลิก Job	18
6. รายละเอียด Singularity Image และ GPU	19
รายละเอียด Singularity Image	19
รายละเอียด NVIDIA A100 GPU	19
7. คู่มือการใช้งาน Singularity	20
7.1 การสร้าง Singularity Image ด้วย Fakeroot feature	20
7.2 การ Pull Container ที่ต้องการใช้งาน	22
7.2.1 กรณีมี container image ที่พร้อมใช้งานอยู่แล้ว	22
7.3 การรัน Container	22
7.3.1 การรัน Singularity Container (CPU-based)	22
7.3.2 การรัน Singularity GPU-Enabled Containers	23
7.4 การเรียกใช้งาน Singularity ผ่าน Slurm	23
8. การใช้งาน Rstudio Server ร่วมกับ Slurm	24

8.1 Login ไปยัง Login node	24
8.2 ตรวจสอบสถานะของ Slurm	24
8.3 การใช้งาน Rstudio Server ผ่าน SLURM	26
การใช้งาน Rstudio Server	26
1) Start Rstudio Server Batch Job ผ่าน Slurm	27
2) Start SSH Tunnel	29
3) เข้าใช้งานผ่าน Browser บนเครื่องของผู้ใช้งาน	32
การเพิ่ม Package Dependencies สำหรับ Library บน Rstudio	33
การหยุดใช้งาน Rstudio server	34
9. การใช้งาน Jupyter notebook บน TensorFlow Singularity image ร่วมกับ Slurm	35
9.1 Login ไปยัง Login node	35
9.2 ตรวจสอบสถานะของ Slurm	35
9.3 การใช้งาน Jupyter notebook บน TensorFlow Singularity image ร่วมกับ Slurm	36
การใช้งาน Jupyter notebook บน TensorFlow	36
1) Start Jupyter notebook Batch Job ผ่าน Slurm	37
2) Start SSH Tunnel	39
3) เข้าใช้งานผ่าน Browser บนเครื่องของผู้ใช้งาน	41
การหยุดใช้งาน Jupyter notebook บน TensorFlow Singularity Image	42

1. รายละเอียด IP Address

No.	Hostname	Public Network	Private Network	Storage Network
1	cuaim-frontend1	161.200.107.77	172.76.0.250	172.76.1.250
2	cuaim-fat-01	-	172.76.0.1	172.76.1.1
3	cuaim-dgx-01	-	172.76.0.2	172.76.1.2

2. รายละเอียดพื้นที่ใช้งานใน CUAIM Cluster

พื้นที่ใช้งานหลักจะอยู่บนระบบไฟล์แบบขนาน Spectrum Scale (GPFS) ที่ mount point ชื่อ /data มีขนาดทั้งสิ้น 450TB ดังภาพ

```
[root@cuaim-frontend1 ~]# df -h
Filesystem      Size  Used Avail Use% Mounted on
devtmpfs        32G   0    32G   0% /dev
tmpfs           32G   0    32G   0% /dev/shm
tmpfs           32G  395M   31G   2% /run
tmpfs           32G   0    32G   0% /sys/fs/cgroup
/dev/mapper/centos-root 190G  11G  179G   6% /
/dev/sda2       1014M  170M  845M  17% /boot
/dev/sda1       200M   12M  189M   6% /boot/efi
tmpfs           6.3G   48K   6.3G   1% /run/user/0
data            450T  350G  450T   1% /data
```

โดยไดเรกทอรีภายใน ประกอบด้วย

Path	รายละเอียด
/data/apps/	พื้นที่เก็บข้อมูลส่วนแอปพลิเคชันต่าง ๆ
/data/container_image/	พื้นที่เก็บข้อมูลส่วน container image สำหรับ Singularity
/data/home/	พื้นที่เก็บข้อมูลส่วนบุคคลของผู้ใช้งาน
/data/scratch/	พื้นที่หลักสำหรับใช้งาน การรันโปรแกรม การเก็บข้อมูลหรือผลลัพธ์ที่ได้จากการทำงาน เป็นต้น

```
[root@cuaim-frontend1 data]# ll
total 3
drwxr-xr-x 2 root root 4096 Feb 27 13:51 apps
drwxr-xr-x 2 root root 4096 Mar  2 12:06 container_image
drwxr-xr-x 7 root root 4096 Mar  5 12:04 home
drwxr-xr-x 2 root root 4096 Feb  3 18:12 ipa-data
drwxrwxrwx 4 root root 4096 Mar  5 10:30 scratch
drwxr-xr-x 8 root root 4096 Mar  5 10:26 utils
```

ตารางแสดงรายละเอียดพื้นที่ใช้งาน ตาม Directory ต่างๆ

Disk Quota			
No	Name	Mount Point	Size
1	Default home	/data/users/<user>	200GB
2	Default Scratch	/data/scratch/<user>/<jobid>	
3	Deepmed	/data/deepmed	40GB
4	Project: cuaim	/data/project/cuaim	100TB
5	Project: cubig	/data/project/cubig	15TB
6	Project: cccn	/data/project/cccn	5TB
7	Project: chics	/data/project/chics	5TB
8	Project: ekapol	/data/project/ekapol	5TB
9	Project: voraphoj	/data/project/voraphoj	10TB
10	Project: pornchai	/data/project/pornchai	5TB
11	cgen	/data/cgen	10GB

3. รายละเอียด User

No.	Username	First name	Group
1	korapint	korapint	cuaim
2	kantam	kantam	cuaim
3	supichayaa	supichayaa	cuaim
4	sirasris	sirasris	cuaim
5	thanakorn	thanakorn	cuaim
6	natthananr	natthananr	cuaim
7	tpakornchote	tpakornchote	chics
8	thanitt	thanitt	ekapol
9	voraphojlab	voraphojlab	voraphoj
10	voraphojn	voraphojn	voraphoj
11	jirachotek	jirachotek	pornchai
12	thanac	thanac	-
13	dhupb	dhupb	cuaim
14	chawanp	chawanp	cuaim
15	cgen	cgen	-
16	korntharak	korntharak	voraphoj
17	songthama	songthama	voraphoj

รหัสผ่านสำหรับการเข้าใช้งานครั้งแรก คือ CU@!Mhpc22

[โดยระบบจะบังคับให้เปลี่ยนรหัสผ่านในการเข้าใช้งานครั้งแรก](#)

4. การใช้งาน CUAIM Cluster ผ่านโปรแกรม SSH

1) กรณีเชื่อมต่อจากภายนอก ให้ทำการ Login เข้าไปยัง VPN ของมหาวิทยาลัย ก่อน ให้สามารถเข้าถึง Network วง 161.200.107.0 ได้

2) ใช้งานไปยัง CUAIM Cluster ด้วยการ Secure Shell ผ่านทางเครื่อง cuaim-frontend1 (IP: 161.200.107.77) โดยตัวอย่างในภาพด้านล่าง ใช้โปรแกรม Terminal เชื่อมต่อไปยังระบบ โดยกรอกข้อมูลในช่อง Host Name (or IP address) ในรูปแบบของ <username>@cuaim-frontend1 เช่น peeranon@cuaim-frontend1

```
[root@frontend1 ~]# ssh peeranon@cuaim-frontend1-10g
Password:
Last login: Sat Mar 5 12:26:23 2022 from localhost
[peeranon@cuaim-frontend1 ~]$ sinfo
```

3) ผู้ใช้งานจะสามารถ SSH ไปยังเครื่อง compute node โดยตรงได้ ก็ต่อเมื่อมีการรันงานบน Slurm เท่านั้น ในกรณีที่ไม่มีงานรันอยู่ เมื่อทำการ SSH ไปที่ Compute node จะแสดงผลดังภาพด้านล่าง

```
[peeranon@cuaim-frontend1 ~]$ ssh cuaim-fat-01
Access denied by pam_slurm_adopt: you have no active jobs on this node
Authentication failed.
[peeranon@cuaim-frontend1 ~]$ ssh cuaim-dgx-01
Access denied by pam_slurm_adopt: you have no active jobs on this node
Authentication failed.
```

5. การใช้งาน Slurm Workload Manager

SLURM Workload Manager คือ ระบบจัดสรรทรัพยากรในระบบ

1) การใช้งาน Slurm

ทางทีมงานใช้โปรแกรม Terminal บน MobaXTerm ในการสาธิตการใช้งาน

1. SSH ไปยังเครื่อง cuaim-frontend1 ด้วย user ของผู้ใช้งาน เช่นในตัวอย่าง ทำการ SSH ไปยัง IP: 161.200.107.77 ซึ่งเป็นเครื่อง cuaim-frontend1

2. เมื่อ Login สำเร็จ จะเข้ามายังเครื่อง Frontend จึงจะสามารถใช้งาน Slurm ได้

```
[root@frontend1 ~]# ssh peeranon@cuaim-frontend1-10g
Password:
Last login: Sat Mar 5 12:26:23 2022 from localhost
[peeranon@cuaim-frontend1 ~]$ sinfo
```

2) การตรวจสอบสถานะต่าง ๆ ภายในระบบ

```
[root@cuaim-frontend1 ~]# sinfo
PARTITION      AVAIL  TIMELIMIT  NODES  STATE NODELIST
cpu*            up    infinite     1    mix  cuaim-fat-01
gpu             up    infinite     1    idle cuaim-dgx-01
interactive-cpu up    infinite     1    mix  cuaim-fat-01
interactive-gpu up    infinite     1    idle cuaim-dgx-01
[root@cuaim-frontend1 ~]#
```

ใช้คำสั่ง `sinfo` สำหรับตรวจสอบสถานะต่าง ๆ ในระบบ โดยคำสั่งจะแสดงข้อมูลดังนี้

- PARTITION แสดงถึงชื่อ Partition ซึ่งทำหน้าที่เปรียบเสมือน Queue สำหรับรับงานเข้าในระบบ โดย Partition ที่มีดอกจัน (*) หมายถึง เป็น Default Partition
- AVAIL แสดงถึงสถานะของ Partition ว่า up หรือไม่
- TIMELIMIT แสดงถึงเวลาสูงสุดที่งานจะสามารถรันได้
- NODES แสดงถึงจำนวนโหนดใน Partition โดยชื่อโหนดจะแสดงอยู่ในคอลัมน์ NODELIST
- STATE แสดงถึงสถานะของ Partition โดยมีค่าต่าง ๆ ดังนี้
 - idle หมายถึง ทุกเครื่องใน Partition ไม่มีงานรันอยู่ ทรัพยากรทั้งหมดยังไม่ถูกจอง
 - mix หมายถึง มีการจองทรัพยากรไว้ ในขณะที่เดียวกันก็ยังมีทรัพยากรว่างอยู่ด้วย
 - CG หรือ Completing หมายถึง งานที่รันไว้กำลังจะเสร็จสิ้น
 - Down หมายถึง Partition นี้ ไม่สามารถใช้งานได้
 - Draining หมายถึง โหนดกำลังรันงาน แต่ไม่สามารถรับงานอื่น ๆ เพิ่มเติมได้
 - Unknown หมายถึง ไม่สามารถระบุสถานะของโหนดใน Partition ได้
- NODELIST แสดงถึงชื่อของโหนดที่อยู่ใน Partition ดังกล่าว

ตาราง Partitions

Partitions					
No	Partition	Nodes	AllowQOS	Limits	Walltime
1	cpu	cuaim-fat-01	normal,cpu32,cpu16,cpu8,cpu4,cpu24h,ngs8,ngs16,studentcpu	Up to QOS**	Up to QOS**
2	gpu	cuaim-dgx-01	normal,gpu40g,gpu20g,gpu10g,gpu5g,studentgpu	Up to QOS**	Up to QOS**
3	interactive-cpu	cuaim-fat-01	interactive-cpu*	cpu=2, mem=8G	2 hr.
4	interactive-gpu	cuaim-dgx-01	interactive-gpu*	cpu=4, mem=16G, gres=gpu:1g.5gb:1	2 hr.

* ผู้ใช้งานทุกคนจะสามารถใช้ QOS interactive-cpu และ interactive-gpu ได้

** Limits ด้านทรัพยากรและ Walltime ขึ้นอยู่กับ QOS ของผู้ใช้งานแต่ละคน

จากตาราง Partitions ด้านบน ข้อมูลที่ผู้ใช้งานจำเป็นต้องทราบ ได้แก่

- ช่อง AllowQOS แสดงถึง QOS ที่จะสามารถ submit งานเข้าสู่ Partition ได้
- ช่อง Limits แสดงถึงทรัพยากร CPU, Memory หรือ GPU ที่ผู้ใช้งานจะสามารถใช้ได้ ณ ขณะนั้น
- ช่อง Walltime แสดงถึงระยะเวลาสูงสุดที่สามารถรันงานได้

1. “cpu” partition

- a. มีโหนด cuaim-fat-01 เป็นสมาชิกอยู่
- b. QOS ที่สามารถใช้งานได้ ได้แก่ normal, cpu32, cpu16, cpu8, cpu4, cpu24h, ngs8, ngs16 และ studentcpu
- c. Limits และ Walltime ตรวจสอบได้จากตาราง QOS

2. “gpu” partition

- a. มีโหนด cuaim-dgx-01 เป็นสมาชิกอยู่
- b. QOS ที่สามารถใช้งานได้ ได้แก่ normal, gpu40g, gpu20g, gpu10g, gpu5g และ studentgpu
- c. Limits และ Walltime ตรวจสอบได้จากตาราง QOS

3. “interactive-cpu” partition

- a. มีโหนด cuaim-fat-01 เป็นสมาชิกอยู่
- b. QOS ที่สามารถใช้งานได้ ได้แก่ interactive-cpu
- c. Limits
 - i. CPU = 2 cores

- ii. Memory = 8 GB
- d. Walltime = 2 ชั่วโมง
- 4. “interactive-gpu” partition
 - a. มีโหนด cuaim-dgx-01 เป็นสมาชิกอยู่
 - b. QOS ที่สามารถใช้งานได้ ได้แก่ interactive-gpu
 - c. Limits
 - i. CPU = 4 cores
 - ii. Memory = 16 GB
 - iii. GPU = gpu:1g.5gb จำนวน 1 หน่วย
 - d. Walltime = 2 ชั่วโมง

ผู้ใช้งานสามารถตรวจสอบ Resource ทั้งหมด ในแต่ละ Partition เพิ่มเติม ได้จากคำสั่ง `check_resource` อีกด้วย

```
[root@cuaim-frontend1 ~]# check_resource
sinfo
PARTITION    AVAIL  TIMELIMIT  NODES  STATE NODELIST
cpu*         up      infinite    1      mix cuaim-fat-01
gpu          up      infinite    1      mix cuaim-dgx-01
interactive-cpu up      infinite    1      mix cuaim-fat-01
interactive-gpu up      infinite    1      mix cuaim-dgx-01

sinfo -o "%15R %8D %10m %5c %7z %60G %N"
PARTITION    NODES    MEMORY    CPUS S:C:T    GRES                                NODELIST
cpu          1        773339    72 4:18:1  (null)                             cuaim-fat-01
gpu          1        1031912   128 2:64:1  gpu:7g.40gb:4,gpu:3g.20gb:5,gpu:2g.10gb:3,gpu:1g.5gb:4 cuaim-dgx-01
interactive-cpu 1        773339    72 4:18:1  (null)                             cuaim-fat-01
interactive-gpu 1        1031912   128 2:64:1  gpu:7g.40gb:4,gpu:3g.20gb:5,gpu:2g.10gb:3,gpu:1g.5gb:4 cuaim-dgx-01
[root@cuaim-frontend1 ~]#
```

3) การตรวจสอบสถานะงานภายในคิว

```
[peeranon@cuaim-frontend1 ~]$ squeue
JOBID PARTITION NAME USER ST TIME NODES NODELIST(REASON)
```

ใช้คำสั่ง `squeue` เพื่อตรวจสอบสถานะงานภายในคิว ก่อนสั่งงานจริง โดยมีรายละเอียดดังนี้

- JOBID แสดงถึง เลข Job
- PARTITION แสดงถึง Partition ที่ Job นี้ทำงานอยู่
- NAME แสดงถึง ชื่อ Job
- USER แสดงถึง ชื่อ User ที่สั่งรัน
- ST คือ State ของงาน โดยสถานะต่าง ๆ มีดังนี้

Status	Code	Explanation
COMPLETED	CD	The job has been completed successfully.
COMPLETING	CG	The job is finished but some processes are still active.
FAILED	F	The job terminated with a non-zero exit code and failed to execute.
PENDING	PD	The job is waiting for resource allocation. It will eventually run.
RUNNING	R	The job is currently allocated to a node and is running.

4) การตรวจสอบทรัพยากรที่ใช้งานได้ทั้งหมด

คำสั่ง `scontrol` สามารถใช้ตรวจสอบข้อมูลต่าง ๆ ใน Slurm ได้ เช่น nodes, partitions, jobs โดยคำสั่ง

`scontrol show nodes` ใช้ตรวจสอบข้อมูลของโหนดต่าง ๆ ภายใน Slurm Cluster

```
[root@cuaim-frontend1 data]# scontrol show nodes
NodeName=cuaim-dgx-01 Arch=x86_64 CoresPerSocket=64
CPUAlloc=0 CPUTot=128 CPULoad=0.74
AvailableFeatures=(null)
ActiveFeatures=(null)
Gres=gpu:8(S:0-1)
NodeAddr=cuaim-dgx-01 NodeHostName=cuaim-dgx-01 Version=21.08.5
OS=Linux 5.4.0-81-generic #91-Ubuntu SMP Thu Jul 15 19:09:17 UTC 2021
RealMemory=1031912 AllocMem=0 FreeMem=1008540 Sockets=2 Boards=1
State=IDLE ThreadsPerCore=1 TmpDisk=0 Weight=1 Owner=N/A MCS_label=N/A
Partitions=gpu
BootTime=2022-03-01T23:20:07 SlurmdStartTime=2022-03-02T10:58:58
LastBusyTime=2022-03-05T12:17:39
CfgTRES=cpu=128,mem=1031912M,billing=128,gres/gpu=8
AllocTRES=
CapWatts=n/a
CurrentWatts=0 AveWatts=0
ExtSensorsJoules=n/s ExtSensorsWatts=0 ExtSensorsTemp=n/s

NodeName=cuaim-fat-01 Arch=x86_64 CoresPerSocket=18
CPUAlloc=0 CPUTot=72 CPULoad=0.01
AvailableFeatures=(null)
ActiveFeatures=(null)
Gres=(null)
NodeAddr=cuaim-fat-01 NodeHostName=cuaim-fat-01 Version=21.08.5
OS=Linux 3.10.0-1160.el7.x86_64 #1 SMP Mon Oct 19 16:18:59 UTC 2020
RealMemory=773339 AllocMem=0 FreeMem=680807 Sockets=4 Boards=1
State=IDLE ThreadsPerCore=1 TmpDisk=0 Weight=1 Owner=N/A MCS_label=N/A
Partitions=cpu
BootTime=2022-03-01T16:53:42 SlurmdStartTime=2022-03-01T23:30:51
LastBusyTime=2022-03-05T12:17:39
```

หรือใช้คำสั่ง `check_resource`

```
[root@cuaim-frontend1 ~]# check_resource
sinfo
PARTITION      AVAIL    TIMELIMIT  NODES  STATE MODELIST
cpu*           up        infinite    1      mix cuaim-fat-01
gpu           up        infinite    1      mix cuaim-dgx-01
interactive-cpu up        infinite    1      mix cuaim-fat-01
interactive-gpu up        infinite    1      mix cuaim-dgx-01

sinfo -o "%15R %8D %10m %5c %7z %60G %N"
PARTITION  NODES  MEMORY  CPUS S:C:T  Gres                                MODELIST
cpu        1      773339   72 4:18:1  (null)                             cuaim-fat-01
gpu        1      1031912 128 2:64:1  gpu:7g.40gb:4,gpu:3g.20gb:5,gpu:2g.10gb:3,gpu:1g.5gb:4 cuaim-dgx-01
interactive-cpu 1      773339   72 4:18:1  (null)                             cuaim-fat-01
interactive-gpu 1      1031912 128 2:64:1  gpu:7g.40gb:4,gpu:3g.20gb:5,gpu:2g.10gb:3,gpu:1g.5gb:4 cuaim-dgx-01
[root@cuaim-frontend1 ~]#
```

CPU, Memory Resources			
No	Node	CPU (Cores)	Memory
1	cuaim-fat-01	72	768 GB
2	cuaim-dgx-01	128	1 TB

GPU Resources		
No	GPU Profile Name	Amount
1	7g.40gb	4
2	3g.20gb	5
3	2g.10gb	3
4	1g.5gb	4

5) การตรวจสอบ QOS ในระบบ

ตารางแจกแจงทรัพยากรในแต่ละ QOS เป็นดังนี้

QoS							
No	Name	Max CPU Per User (Cores)	Max RAM Per User (GB)	Max GPU Per User (Unit)	Max Walltime Per User (Day-Hour:Min:Sec)	Partition	Members
1	normal	Unlimited	Unlimited	Unlimited	Unlimited	cpu, gpu	cgen, kantam, korapint, supichayaa
2	gpu40g	16	64	7g.40gb=1	0-08:00:00	gpu	chawanp, natthananr, dhupb, thanitt
3	gpu20g	8	32	3g.20gb=1	0-08:00:00	gpu	chawanp, natthananr, dhupb, korntharak, songthama, voraphojn, thanac
4	gpu10g	4	16	2g.10gb=1	0-04:00:00	gpu	sirasris, jirachotek
5	gpu5g	2	8	1g.5gb=1	0-04:00:00	gpu	
6	cpu32	32	8	-	0-04:00:00	cpu	
7	cpu16	16	8	-	0-04:00:00	cpu	
8	cpu8	8	8	-	0-04:00:00	cpu	sirasris
9	cpu4	4	8	-	0-04:00:00	cpu	
10	cpu24H	24	256	-	0-04:00:00	cpu	thanakorn
11	ngs16	16	96	-	0-08:00:00	cpu	
12	ngs8	8	48	-	0-08:00:00	cpu	
13	studentcpu	2	8	-	0-02:00:00	cpu	
14	studentgpu	2	8	1g.5gb=1	0-02:00:00	gpu	
15	interactive-cpu	2	8	-	0-02:00:00	interactive-cpu	All users
16	interactive-gpu	2	8	1g.5gb=1	0-02:00:00	interactive-gpu	All users

ใช้คำสั่ง check_qos

```
[root@cuaim-frontend1 ~]# check_qos
sacctmgr show qos format=name%15,MaxTRESPU%90,MaxWall
-----
```

Name	MaxTRESPU	MaxWall
normal		
chics		
ekapol		
voraphoj		
pornchai		
thanac		
cuaim		
gpu40g	cpu=16, gres/gpu:1g.5gb=0, gres/gpu:2g.10gb=0, gres/gpu:3g.20gb=0, gres/gpu:7g.40gb=1, mem=64G	8-00:00:00
gpu20g	cpu=8, gres/gpu:1g.5gb=0, gres/gpu:2g.10gb=0, gres/gpu:3g.20gb=1, gres/gpu:7g.40gb=0, mem=32G	8-00:00:00
gpu10g	cpu=4, gres/gpu:1g.5gb=0, gres/gpu:2g.10gb=1, gres/gpu:3g.20gb=0, gres/gpu:7g.40gb=0, mem=16G	4-00:00:00
gpu5g	cpu=2, gres/gpu:1g.5gb=1, gres/gpu:2g.10gb=0, gres/gpu:3g.20gb=0, gres/gpu:7g.40gb=0, mem=8G	4-00:00:00
cpu32	cpu=32, mem=8G	4-00:00:00
cpu16	cpu=16, mem=8G	4-00:00:00
cpu8	cpu=8, mem=8G	4-00:00:00
cpu4	cpu=4, mem=8G	4-00:00:00
cpu24h	cpu=24, mem=256G	4-00:00:00
ngs16	cpu=16, mem=96G	8-00:00:00
ngs8	cpu=8, mem=48G	8-00:00:00
studentgpu	cpu=2, gres/gpu:1g.5gb=1, gres/gpu:2g.10gb=0, gres/gpu:3g.20gb=0, gres/gpu:7g.40gb=0, mem=8G	2-00:00:00
studentcpu	cpu=2, mem=8G	2-00:00:00
interactive-cpu	cpu=2, mem=8G	02:00:00
interactive-gpu	cpu=4, gres/gpu:1g.5gb=1, gres/gpu:2g.10gb=0, gres/gpu:3g.20gb=0, gres/gpu:7g.40gb=0, mem=16G	02:00:00

```
[root@cuaim-frontend1 ~]#
```

- ค่า MaxTRESPU = Max TRES Per User แสดงถึงจำนวนทรัพยากรที่ผู้ใช้งานสามารถจองได้ ณ เวลานั้น
- ค่า MaxWall = ระยะเวลาสูงที่สุดที่สามารถรันงานได้

สามารถตรวจสอบ QOS ที่ User สามารถใช้งานได้ ด้วยคำสั่ง check_group

```
[root@cuaim-frontend1 ~]# check_group
sacctmgr show assoc format=account,user%15,GrpTRES%75,qos%40
-----
```

Account	User	GrpTRES	QOS
root	root		normal
cuaim	cuaim	cpu=50, gres/gpu:2g.10gb=2, gres/gpu:3g.20gb=2, gres/gpu:7g.40gb=2, mem=256G	normal
cuaim	aumsp		cuaim
cuaim	cgen		cpu8, interactive-cpu
cuaim	chawanp		cuaim, interactive-cpu, interactive-gpu, n+
cuaim	dhupb		gpu20g, gpu40g, interactive-cpu, interactive-gpu, n+
cuaim	georgig		gpu20g, gpu40g, interactive-cpu, interactive-gpu, n+
cuaim	kantam		cpu16, interactive-cpu
cuaim	korapint		cuaim, interactive-cpu, interactive-gpu, n+
cuaim	malik		cuaim, interactive-cpu, interactive-gpu, n+
cuaim	natthanar		cpu32, gpu20g, interactive-cpu, interactive-gpu, n+
cuaim	peeranon		gpu20g, gpu40g, interactive-cpu, interactive-gpu, n+
cuaim	sirasris		cpu8, gpu40g, interactive-cpu, interactive-gpu, n+
cuaim	supichayaa		cpu8, gpu10g, interactive-cpu, interactive-gpu, n+
cuaim	thanakorn		cuaim, interactive-cpu, interactive-gpu, n+
g_chics	g_chics		cpu24h, interactive-cpu, interactive-gpu
g_chics	tpakornchote		chics
g_ekapol	g_ekapol		chics, interactive-cpu, interactive-gpu, n+
g_ekapol	thanitt		ekapol
g_kanitha	surapunt		gpu40g, interactive-cpu, interactive-gpu
g_kanitha	g_kanitha		normal
g_pornchai	g_pornchai		interactive-cpu, ngs8
g_pornchai	jirachotek		pornchai
g_thanac	g_thanac		gpu10g, interactive-cpu, interactive-gpu
g_thanac	thanac		thanac
g_voraphoj	g_voraphoj		gpu20g, interactive-cpu, interactive-gpu
g_voraphoj	korntharak	cpu=16, gres/gpu:3g.20gb=2, mem=64G	voraphoj
g_voraphoj	songthama		gpu20g, interactive-cpu, interactive-gpu
g_voraphoj	voraphojn		gpu20g, interactive-cpu, interactive-gpu

```
[root@cuaim-frontend1 ~]#
```

สามารถระบุ user เพื่อแสดงเฉพาะ user ตัวเอง sacctmgr show assoc user=<username>

format=account,user%15,qos%50,defaultqos%20

```
[root@cuaim-frontend1 ~]# sacctmgr show assoc user=peeranon format=account,user%15,qos%50,defaultqos%20
-----
```

Account	User	QOS	Def QOS
cuaim	peeranon	cpu8, gpu40g, interactive-cpu, interactive-gpu	

```
[root@cuaim-frontend1 ~]#
```

เมื่อรัน scontrol show partitions จะพบกับ QOS ที่ User แต่ละคนจะสามารถใช้งานได้

```
[root@cuaaim-frontendl ~]# scontrol show partitions
PartitionName=cpu
  AllowGroups=ALL AllowAccounts=ALL AllowQos=normal,cpu32,cpu16,cpu8,cpu4,cpu24h,ngs8,ngs16,studentcpu
  AllocNodes=ALL Default=YES QoS=N/A
  DefaultTime=NONE DisableRootJobs=NO ExclusiveUser=NO GraceTime=0 Hidden=NO
  MaxNodes=UNLIMITED MaxTime=UNLIMITED MinNodes=0 LLN=NO MaxCPUsPerNode=UNLIMITED
  Nodes=cuaaim-fat-01
  PriorityJobFactor=1 PriorityTier=1 RootOnly=NO ReqResv=NO OverSubscribe=NO
  OverTimeLimit=NONE PreemptMode=OFF
  State=UP TotalCPUs=72 TotalNodes=1 SelectTypeParameters=NONE
  JobDefaults=(null)
  DefMemPerCPU=4096 MaxMemPerNode=UNLIMITED

PartitionName=gpu
  AllowGroups=ALL AllowAccounts=ALL AllowQos=normal,gpu40g,gpu20g,gpu10g,gpu5g,studentgpu
  AllocNodes=ALL Default=NO QoS=N/A
  DefaultTime=NONE DisableRootJobs=NO ExclusiveUser=NO GraceTime=0 Hidden=NO
  MaxNodes=UNLIMITED MaxTime=UNLIMITED MinNodes=0 LLN=NO MaxCPUsPerNode=UNLIMITED
  Nodes=cuaaim-dgx-01
  PriorityJobFactor=1 PriorityTier=1 RootOnly=NO ReqResv=NO OverSubscribe=NO
  OverTimeLimit=NONE PreemptMode=OFF
  State=UP TotalCPUs=128 TotalNodes=1 SelectTypeParameters=NONE
  JobDefaults=(null)
  DefMemPerCPU=4096 MaxMemPerNode=UNLIMITED

PartitionName=interactive-cpu
  AllowGroups=ALL AllowAccounts=ALL AllowQos=interactive-cpu
  AllocNodes=ALL Default=NO QoS=N/A
  DefaultTime=NONE DisableRootJobs=NO ExclusiveUser=NO GraceTime=0 Hidden=NO
  MaxNodes=UNLIMITED MaxTime=UNLIMITED MinNodes=0 LLN=NO MaxCPUsPerNode=UNLIMITED
  Nodes=cuaaim-fat-01
  PriorityJobFactor=1 PriorityTier=1 RootOnly=NO ReqResv=NO OverSubscribe=NO
  OverTimeLimit=NONE PreemptMode=OFF
  State=UP TotalCPUs=72 TotalNodes=1 SelectTypeParameters=NONE
  JobDefaults=(null)
  DefMemPerCPU=1024 MaxMemPerNode=UNLIMITED

PartitionName=interactive-gpu
  AllowGroups=ALL AllowAccounts=ALL AllowQos=interactive-gpu
  AllocNodes=ALL Default=NO QoS=N/A
  DefaultTime=NONE DisableRootJobs=NO ExclusiveUser=NO GraceTime=0 Hidden=NO
  MaxNodes=UNLIMITED MaxTime=UNLIMITED MinNodes=0 LLN=NO MaxCPUsPerNode=UNLIMITED
  Nodes=cuaaim-dgx-01
  PriorityJobFactor=1 PriorityTier=1 RootOnly=NO ReqResv=NO OverSubscribe=NO
  OverTimeLimit=NONE PreemptMode=OFF
  State=UP TotalCPUs=128 TotalNodes=1 SelectTypeParameters=NONE
  JobDefaults=(null)
  DefMemPerCPU=2048 MaxMemPerNode=UNLIMITED
```

6) การ Submit Job

ใช้คำสั่ง `check_resource` เพื่อตรวจสอบทรัพยากรที่มีได้

```
[root@cuaaim-frontendl ~]# check_resource
sinfo
PARTITION    AVAIL    TIMELIMIT  NODES  STATE NODELIST
cpu*         up       infinite   1      mix   cuaaim-fat-01
gpu          up       infinite   1      mix   cuaaim-dgx-01
interactive-cpu up       infinite   1      mix   cuaaim-fat-01
interactive-gpu up       infinite   1      mix   cuaaim-dgx-01

sinfo -o "%15R %8D %10m %5c %7z %60G %N"
PARTITION    NODES    MEMORY    CPUS S:C:T    GRES                                NODELIST
cpu          1        773339    72 4:18:1    (null)                             cuaaim-fat-01
gpu          1        1031912   128 2:64:1    gpu:7g.40gb:4,gpu:3g.20gb:5,gpu:2g.10gb:3,gpu:1g.5gb:4 cuaaim-dgx-01
interactive-cpu 1        773339    72 4:18:1    (null)                             cuaaim-fat-01
interactive-gpu 1        1031912   128 2:64:1    gpu:7g.40gb:4,gpu:3g.20gb:5,gpu:2g.10gb:3,gpu:1g.5gb:4 cuaaim-dgx-01
[root@cuaaim-frontendl ~]#
```

Slurm ได้แบ่งประเภทของงานออกเป็นหลายประเภทด้วยกัน ได้แก่

6.1) Batch job

การส่งร้งงาน Batch สามารถทำได้สองวิธี ทั้ง รันผ่าน Command Line และผ่าน Batch Script (ในที่นี้จะยกตัวอย่างแค่การ Submit Job ผ่าน Command Line)

- Submit ผ่านคำสั่งบน Command Line

```
[peeranon@cuaim-frontend1 ~]$ srun --qos=cpu8 -c1 --mem=1G hostname
cuaim-fat-01.cuaim.hpc
```

```
[peeranon@cuaim-frontend1 ~]$ srun --qos=gpu40g -p gpu -c1 --mem=1G --gres=gpu:7g.40gb:1 nvidia-smi -L
GPU 0: NVIDIA A100-SXM4-40GB (UUID: GPU-41c0004c-792f-9231-481c-96acfac687e1)
GPU 1: NVIDIA A100-SXM4-40GB (UUID: GPU-5ba48061-73d7-68f5-c2b5-3fe1d41787a9)
GPU 2: NVIDIA A100-SXM4-40GB (UUID: GPU-be05d4b7-69e5-4ce7-f504-dal6bfdb568a)
GPU 3: NVIDIA A100-SXM4-40GB (UUID: GPU-82cb75c9-59f1-e184-b501-cc559c8d5073)
MIG 7g.40gb Device 0: (UUID: MIG-7e75633d-b276-511c-92ef-f8d77695783e)
GPU 4: NVIDIA A100-SXM4-40GB (UUID: GPU-b0fc60a3-9b11-e6d1-8022-4abb184599b3)
GPU 5: NVIDIA A100-SXM4-40GB (UUID: GPU-472ad957-6605-bc15-09f4-ecc3afba0fe2)
GPU 6: NVIDIA A100-SXM4-40GB (UUID: GPU-5989ddde-a91e-a5d6-8985-8398adc205a5)
GPU 7: NVIDIA A100-SXM4-40GB (UUID: GPU-14554570-e7cc-ca3b-f394-afc3fc4ba63a)
[peeranon@cuaim-frontend1 ~]$
```

ผู้ใช้งานสามารถส่งรันงานได้ด้วยคำสั่ง **srun** ดังตัวอย่างในภาพด้านบน โดยมีรายละเอียดดังนี้

Option	คำอธิบาย
--qos=gpu40g	ระบุ QOS ของผู้ใช้งาน
-c 1	จำนวน core ที่ต้องการใช้
--mem=2G	ระบุจำนวน Memory (GB) เช่น 2GB
-p gpu	Partition ที่ต้องการใช้งาน เช่น cpu หรือ gpu
--gres=gpu:7g.40gb:1	ระบุจำนวน gpu ที่ต้องการใช้ (สำหรับ MIG GPU Partition เช่น 7g.40gb, 3g.20g, 2g.10gb หรือ 1g.5gb)

- ตัวอย่างการสร้างและรัน Batch script

```
[peeranon@cuaim-frontend1 scripts]$ cat singularity-gpu-simple-cnn.sh
#!/bin/bash
#SBATCH -p gpu
#SBATCH --gres=gpu:1
#SBATCH -J 1xA100
#SBATCH --mem=16G
#SBATCH --qos=gpu40g

singularity exec --nv /data/container_image/tensorflow_latest-gpu.sif python simple_cnn.py
[peeranon@cuaim-frontend1 scripts]$
```

```
[peeranon@cuaim-frontend1 ~]$ sbatch singularity-gpu-simple-cnn.sh
```

6.2) Interactive job

Interactive job คือ การรัน Interactive job เปรียบเสมือนการทำ SSH Session ไปยังเครื่อง compute นั้น โดยจะเหมาะกับการทดสอบในการรันโปรแกรม ว่าโปรแกรมที่เขียนมาทำงานได้ดีหรือไม่ หรือ งานที่ใช้เวลาในการรันไม่นาน และการรัน interactive job หากทำการปิด Session จะทำการยกเลิกการรันโดยทันที

```
srun -p interactive-cpu -c2 --mem=8G --qos=interactive-cpu --pty bash
[peeranon@cuaim-fat-01 ~]$
```

```
[peeranon@cuaaim-frontend1 ~]$ srun -p interactive-gpu -c2 --mem=8G --gres=gpu:1g.5gb:1 --qos=interactive-gpu --pty bash
peeranon@cuaaim-dgx-01:~$ nvidia-smi -L
GPU 0: NVIDIA A100-SXM4-40GB (UUID: GPU-41c0004c-792f-9231-481c-96acfac687e1)
GPU 1: NVIDIA A100-SXM4-40GB (UUID: GPU-5ba48061-73d7-68f5-c2b5-3fe1d41787a9)
GPU 2: NVIDIA A100-SXM4-40GB (UUID: GPU-be05d4b7-69e5-4ce7-f504-da16bfd568a)
GPU 3: NVIDIA A100-SXM4-40GB (UUID: GPU-82cb75c9-59f1-e184-b501-cc559c8d5073)
GPU 4: NVIDIA A100-SXM4-40GB (UUID: GPU-b0fc60a3-9b11-e6d1-8022-4abb184599b3)
GPU 5: NVIDIA A100-SXM4-40GB (UUID: GPU-472ad957-6605-bc15-09f4-ecc3afba0fe2)
GPU 6: NVIDIA A100-SXM4-40GB (UUID: GPU-5989ddde-a91e-a5d6-8985-8398adc205a5)
MIG 1g.5gb Device 0: (UUID: MIG-a3af2722-51a2-55c1-a676-d09efd7ebf62)
GPU 7: NVIDIA A100-SXM4-40GB (UUID: GPU-14554570-e7cc-ca3b-f394-afc3fc4ba63a)
peeranon@cuaaim-dgx-01:~$
```

การยกเลิกการรันงาน Interactive Job ให้ทำการกด Ctrl + D หรือ พิมพ์ exit เพื่อทำการ terminate ออกจาก Interactive job mode

6.2) Submit job with QOS

ในกรณีที่ User มี QOS ของแต่ละคนแล้ว ในการใช้งานให้เพิ่ม option qos=... ไปในคำสั่ง submit job ด้วย

```
[korntharak@cuaaim-frontend1 ~]$ srun -p gpu -c1 --mem=1G --gres=gpu:3g.20gb:2 nvidia-smi -L
srun: job 715 queued and waiting for resources
^Csrun: Job allocation 715 has been revoked
srun: Force Terminated job 715
[korntharak@cuaaim-frontend1 ~]$ srun -p gpu --qos=gpu20g -c1 --mem=1G --gres=gpu:3g.20gb:1 nvidia-smi -L
GPU 0: NVIDIA A100-SXM4-40GB (UUID: GPU-41c0004c-792f-9231-481c-96acfac687e1)
GPU 1: NVIDIA A100-SXM4-40GB (UUID: GPU-5ba48061-73d7-68f5-c2b5-3fe1d41787a9)
GPU 2: NVIDIA A100-SXM4-40GB (UUID: GPU-be05d4b7-69e5-4ce7-f504-da16bfd568a)
GPU 3: NVIDIA A100-SXM4-40GB (UUID: GPU-82cb75c9-59f1-e184-b501-cc559c8d5073)
GPU 4: NVIDIA A100-SXM4-40GB (UUID: GPU-b0fc60a3-9b11-e6d1-8022-4abb184599b3)
MIG 3g.20gb Device 0: (UUID: MIG-47306b49-d6d5-53fb-86b5-203a4ac7c21d)
GPU 5: NVIDIA A100-SXM4-40GB (UUID: GPU-472ad957-6605-bc15-09f4-ecc3afba0fe2)
GPU 6: NVIDIA A100-SXM4-40GB (UUID: GPU-5989ddde-a91e-a5d6-8985-8398adc205a5)
GPU 7: NVIDIA A100-SXM4-40GB (UUID: GPU-14554570-e7cc-ca3b-f394-afc3fc4ba63a)
[korntharak@cuaaim-frontend1 ~]$
```

7) การตรวจสอบสถานะของงาน

ใช้คำสั่ง `squeue` ในการแสดงสถานะของงาน

```
peeranon@cuaim-dgx-01:~$ squeue -u peeranon
```

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)
53	gpu	bash	peeranon	R	0:47	1	cuaim-dgx-01

โดยมีรายละเอียดดังนี้

- JOBID คือ รหัสของ Job
- PARTITION บอกว่า Job นี้ทำการรันอยู่ที่ Partition กลุ่มไหน
- NAME คือ ชื่อของ Job ที่ทำการระบุไว้ใน การ Submit job
- USER คือ ผู้ใช้งานที่ทำการรัน Job
- ST คือ Status การทำงานของ Job นั้นๆ ยกตัวอย่างเช่น PD = Pending (รอคิว), R = Running (กำลังรันอยู่) เป็นต้น
- TIME คือ เวลาที่ใช้รันงานของ Job
- NODES คือ Job นี้ใช้ จำนวนกี่เครื่องในการรัน
- NODELIST คือ รายชื่อเครื่องที่ Job นี้ ใช้ในการรัน ได้แก่ Compute node

8) การยกเลิก Job

ใช้คำสั่ง `scancel -f <JOB_ID>`

```
[root@cuaim-frontend1 ~]# scancel -f 53
[root@cuaim-frontend1 ~]#
```

6. รายละเอียด Singularity Image และ GPU

รายละเอียด Singularity Image

Singularity images สำหรับผู้ใช้งานทุกคน รวมอยู่ที่ `/data/container_image/`

```
root@cuaim-dgx-01:~# ls /data/container_image/  
tensorflow-keras_20.12-tf2-py3.def tensorflow-keras_20.12-tf2-py3.sif tensorflow_latest-gpu.sif
```

รายละเอียด NVIDIA A100 GPU

```
root@cuaim-dgx-01:~# nvidia-smi -L  
GPU 0: NVIDIA A100-SXM4-40GB (UUID: GPU-41c0004c-792f-9231-481c-96acf687e1)  
MIG 7g.40gb Device 0: (UUID: MIG-3328c4dd-c14d-5cc9-b7b9-f18bf3b585a3)  
GPU 1: NVIDIA A100-SXM4-40GB (UUID: GPU-5ba48061-73d7-68f5-c2b5-3fe1d41787a9)  
MIG 7g.40gb Device 0: (UUID: MIG-b4b228b3-da00-579c-aleb-618cla2f6bef)  
GPU 2: NVIDIA A100-SXM4-40GB (UUID: GPU-be05d4b7-69e5-4ce7-f504-da16bfdb568a)  
MIG 7g.40gb Device 0: (UUID: MIG-dcb42887-dacb-5eaa-9cf2-4268298af7d2)  
GPU 3: NVIDIA A100-SXM4-40GB (UUID: GPU-82cb75c9-59f1-e184-b501-cc559c8d5073)  
MIG 7g.40gb Device 0: (UUID: MIG-7e75633d-b276-511c-92ef-f8d77695783e)  
GPU 4: NVIDIA A100-SXM4-40GB (UUID: GPU-b0fc60a3-9b11-e6d1-8022-4abb184599b3)  
MIG 3g.20gb Device 0: (UUID: MIG-47306b49-d6d5-53fb-86b5-203a4ac7c21d)  
MIG 3g.20gb Device 1: (UUID: MIG-a65d0126-3d22-5890-b7da-14fc566dfcfb)  
GPU 5: NVIDIA A100-SXM4-40GB (UUID: GPU-472ad957-6605-bc15-09f4-ecc3afba0fe2)  
MIG 3g.20gb Device 0: (UUID: MIG-6533fb53-7d0f-5904-a88e-59ff73762bb9)  
MIG 3g.20gb Device 1: (UUID: MIG-f05030a1-3782-5055-a15d-0b7a45f07ccd)  
GPU 6: NVIDIA A100-SXM4-40GB (UUID: GPU-5989ddde-a91e-a5d6-8985-8398adc205a5)  
MIG 3g.20gb Device 0: (UUID: MIG-74729360-8273-59d9-8874-b3385b402cd9)  
MIG 1g.5gb Device 1: (UUID: MIG-a3af2722-51a2-55c1-a676-d09efd7ebf62)  
MIG 1g.5gb Device 2: (UUID: MIG-0b7a65bf-cbc8-5442-bf2c-630473394227)  
MIG 1g.5gb Device 3: (UUID: MIG-12835531-7616-5125-ae2c-8c0d12100000)  
GPU 7: NVIDIA A100-SXM4-40GB (UUID: GPU-14554570-e7cc-ca3b-f394-afc3fc4ba63a)  
MIG 2g.10gb Device 0: (UUID: MIG-b0753415-6822-5a43-9a73-793a931eccc1)  
MIG 2g.10gb Device 1: (UUID: MIG-d5d64e71-b610-5c36-a1e5-fd39d1dfb984)  
MIG 2g.10gb Device 2: (UUID: MIG-7bc15080-9779-5348-ab23-2f845c7a857d)  
MIG 1g.5gb Device 3: (UUID: MIG-b468ff0e-499c-5376-8a65-e01526aeb481)  
root@cuaim-dgx-01:~#
```

7. คู่มือการใช้งาน Singularity

Singularity คือ ระบบ Container ที่ใช้งานบนระบบ HPC ซึ่งทำงานคล้าย Docker และสามารถนำ Image จาก Docker มารันได้

ในขณะที่รันภายใน Singularity container ผู้ใช้งานมีระดับของสิทธิ์เหมือนกับภายนอก container โดยทั่วไปแล้ว Singularity image จะต้องถูกพัฒนาและสร้างขึ้นมาจากพื้นฐานเครื่อง Linux ซึ่งต้องมีสิทธิ์เทียบเท่า Administrator โดยในระบบ หรือใช้งาน Feature Fakeroot โดยตัวอย่างไฟล์ image และไฟล์ .def สำหรับผู้ใช้งานทุกคนถูกวางไว้ที่

/data/container_image/

```
(base) [peeranon@cuaaim-frontendl ~]$ ls -alh /data/container_image/
total 31G
drwxr-xr-x  2 root root 4.0K Jun 15 10:18 .
drwxr-xr-x 13 root root 256K Jun  3 15:05 ..
-rwxr-xr-x  1 root root 875M Jun 15 10:17 chrstnhntschl_gpu_burn_latest.sif
-rwxr-xr-x  1 root root 420M Mar 17 2022 fl-webapp-front.sif
-rw-r--r--  1 root root 1.3G Mar 17 2022 fl-webapp-front.tar
-rwxr-xr-x  1 root root 1.5G Jun 15 10:12 oguzpastirmaci_gpu-burn_latest.sif
-rwxr-xr-x  1 root root 6.5G Apr 11 2022 pytorch-22.03-py3.sif
-rw-rw-r--  1 root root 217 Apr 19 2022 pytorch-add-camelcase.def
-rwxr-xr-x  1 root root 6.5G Apr 19 2022 pytorch-add-camelcase.sif
-rwxr-xr-x  1 root root 521M Apr  5 2022 rstudio_4.1.2.sif
-rw-r--r--  1 root root 580 May 16 2022 tensorflow-custom-env.def
-rwxr-xr-x  1 root root 5.4G May 16 2022 tensorflow-custom-env.sif
-rw-r--r--  1 root root 369 Mar  2 2022 tensorflow-keras_20.12-tf2-py3.def
-rwxr-xr-x  1 root root 5.4G Mar  2 2022 tensorflow-keras_20.12-tf2-py3.sif
-rwxr-xr-x  1 root root 2.6G Mar  2 2022 tensorflow_latest-gpu.sif
(base) [peeranon@cuaaim-frontendl ~]$
```

7.1 การสร้าง Singularity Image ด้วย Fakeroot feature

ในกรณีที่คุณมีเครื่องที่ติดตั้ง Singularity บนเครื่อง Linux ไว้แล้ว เราก็สามารถที่จะสร้าง Singularity image ได้ โดยสร้าง Singularity Definition file โดยให้มีรายละเอียดดังตัวอย่างต่อไปนี้

```
(base) [peeranon@cuaaim-frontendl ~]$ cat /data/container_image/tensorflow-keras_20.12-tf2-py3.def
Bootstrap: docker
From: nvcr.io/nvidia/tensorflow:20.12-tf2-py3

%runscript
    echo "Singularity image with TensorFlow 2 and Keras running on python 3.5 with GPU support."

%post
    # Set up some e required environment defaults
    export LC_ALL=C
    export PATH=/bin:/sbin:/usr/bin:/usr/sbin:$PATH

    # Install other commonly-needed packages
    pip install -U keras

    mkdir /data
```

ภายในไฟล์ เราจำเป็นต้องการให้ Singularity สร้าง image บน Ubuntu 18.04 โดยดึง base image จาก Docker Hub ภายได้ %runscript จะใส่สิ่งที่ต้องการติดตั้งลงไป โดยจะทำเพียงครั้งเดียวในตอนสร้าง image ในกรณีนี้ เราใช้คำสั่ง apt-get ในการติดตั้ง Python 3 พร้อมกับ NumPy และ SciPy และสุดท้ายถ้าเราต้องการให้ image สามารถใช้งานพื้นที่จัดเก็บภายใน Cluster หรือบน NFS เราจำเป็นต้องประกาศ directories ต่าง ๆ ภายใน container ได้แก่ /data ถ้าไม่ประกาศ directories เหล่านี้ภายใน container เราอาจจะได้รับ warning เมื่อรัน container นี้

ในการ Build image สามารถรันคำสั่งด้านล่างนี้ได้เลย พร้อมระบุ flag fakeroot และตำแหน่งของ image และ Definition file

```
singularity build --fakeroot example.sif singularity.def
```

```
(base) [peeranon@cuaim-frontend1 ~]$ singularity build --fakeroot tf-keras.sif /data/container_image/tensorflow-keras_20.12-tf2-py3.def
INFO: Starting build...
Getting image source signatures
Copying blob 6a5697faee43 [=====>-----] 24.0MiB / 27.2MiB
```

Examples

Build from a definition file:

```
singularity build --fakeroot /tmp/test.sif /tmp/test.def
```

Ping from container:

```
singularity exec --fakeroot --net docker://alpine ping -c1 8.8.8.8
```

HTTP server:

```
singularity run --fakeroot --net --network-args="portmap=8080:80/tcp" -w docker://nginx
```

ภาพแสดงตัวอย่างเพิ่มเติมในการใช้ Fakeroot

7.2 การ Pull Container ที่ต้องการใช้งาน

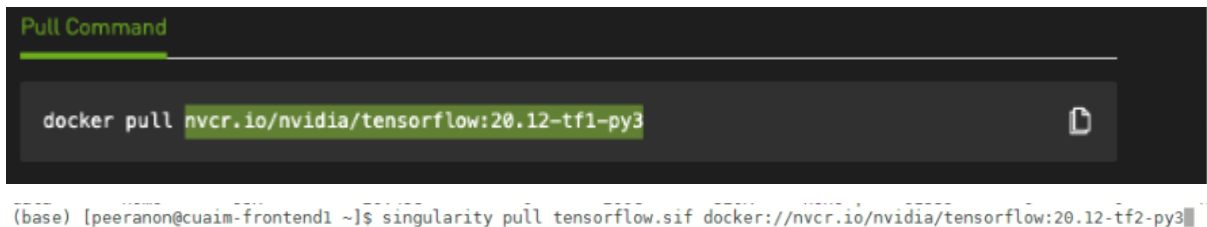
7.2.1 กรณีมี container image ที่พร้อมใช้งานอยู่แล้ว

เลือกแหล่งของ Image เพื่อใช้งานจากเว็บไซต์ที่ต้องการ ตัวอย่างเช่น 2 เว็บไซต์ ได้แก่

- NVIDIA GPU Cloud (NGC) แหล่งรวม Pre-built GPU-Accerelated container
<https://ngc.nvidia.com/catalog/containers>
- Docker hub แหล่งรวม container ของ Docker <https://hub.docker.com/>

```
singularity pull <singularity_name>.sif docker://<pull_url>
```

ในกรณีที่ต้องการ Pull image จาก NGC สามารถนำ Pull URL ที่อยู่ใน Pull Command จากเว็บไซต์ของ NGC มาแทนที่ <pull_url> ในตัวอย่างคำสั่งได้เลย ดังตัวอย่างในภาพด้านล่าง



7.3 การรัน Container

7.3.1 การรัน Singularity Container (CPU-based)

โดยใช้คำสั่งด้านล่าง การใส่ `--nv` เพื่อเป็นการเรียกใช้งาน GPU

```
singularity exec <singularity_name>.sif <command>
```

ตัวอย่างการเรียกใช้งาน Singularity Image

```
(base) peeranon@cuaim-dgx-01:~$ singularity exec /data/container_image/fl-webapp-front.sif cat /etc/os-release
PRETTY_NAME="Debian GNU/Linux 9 (stretch)"
NAME="Debian GNU/Linux"
VERSION_ID="9"
VERSION="9 (stretch)"
VERSION_CODENAME=stretch
ID=debian
HOME_URL="https://www.debian.org/"
SUPPORT_URL="https://www.debian.org/support"
BUG_REPORT_URL="https://bugs.debian.org/"
(base) peeranon@cuaim-dgx-01:~$
```

7.3.2 การรัน Singularity GPU-Enabled Containers

การรัน Singularity เพื่อใช้งาน GPU ให้ใส่ `--nv` ไว้ในคำสั่งเสมอ

```
singularity exec --nv \
/app/singularity_images/tensorflow.sif \
```

python tf_gpu.py

```
(base) peeranon@cuaaim-dgx-01:~$ singularity exec --nv /data/container_image/tensorflow-keras_20.12-tf2-py3.sif nvidia-smi -L
GPU 0: NVIDIA A100-SXM4-40GB (UUID: GPU-41c0004c-792f-9231-481c-96acfac687e1)
GPU 1: NVIDIA A100-SXM4-40GB (UUID: GPU-5ba48061-73d7-68f5-c2b5-3fe1d41787a9)
GPU 2: NVIDIA A100-SXM4-40GB (UUID: GPU-be05d4b7-69e5-4ce7-f504-da16b6db568a)
GPU 3: NVIDIA A100-SXM4-40GB (UUID: GPU-82cb75c9-59f1-e184-b501-cc559c8d5073)
GPU 4: NVIDIA A100-SXM4-40GB (UUID: GPU-b0fc60a3-9b11-e6d1-8022-4abb184599b3)
GPU 5: NVIDIA A100-SXM4-40GB (UUID: GPU-472ad957-6605-bc15-09f4-ecc3afba0fe2)
GPU 6: NVIDIA A100-SXM4-40GB (UUID: GPU-5989ddde-a91e-a5d6-8985-8398adc205a5)
MIG 1g.5gb Device 0: (UUID: MIG-a3af2722-51a2-55c1-a676-d09efd7ebf62)
GPU 7: NVIDIA A100-SXM4-40GB (UUID: GPU-14554570-e7cc-ca3b-f394-afc3fc4ba63a)
(base) peeranon@cuaaim-dgx-01:~$
```

7.4 การเรียกใช้งาน Singularity ผ่าน Slurm

ตัวอย่างการใช้นาคำสั่ง Bash Script ในการเรียกใช้งาน Singularity ใน Slurm Cluster

```
(base) [peeranon@cuaaim-frontend1 scripts]$ cat singularity-gpu-simple-cnn.sh
#!/bin/bash
#SBATCH -p interactive-gpu
#SBATCH -c2
#SBATCH --gres=gpu:1g.5gb:1
#SBATCH -J 1x1g.5gb
#SBATCH --mem=16G
#SBATCH --qos=interactive-gpu

singularity exec --nv /data/container_image/tensorflow_latest-gpu.sif python simple_cnn.py
```

ทำการ Submit งาน ด้วยคำสั่ง sbatch

```
(base) [peeranon@cuaaim-frontend1 ~]$ sbatch scripts/singularity-gpu-simple-cnn.sh
Submitted batch job 5389
```

ภายหลังจากการ Submit สำเร็จ ให้ลองตรวจสอบงานด้วยคำสั่ง squeue เพื่อตรวจสอบสถานะของงาน

```
5389 interacti 1x1g.5gb peeranon R 0:01 1 cuaaim-dgx-01
```

เมื่องานรันเสร็จแล้ว จะพบกับไฟล์ Output แบบ Default name ที่ชื่อ slurm-<JOBID>.out โดย <JOBID> จะเป็นเลข Job ที่เราได้ตอนสั่งรัน sbatch เช่น slurm-5389.out เมื่อลองเปิดไฟล์ดู จะพบกับ Output log จากงานที่เราสั่งรันไปเมื่อสักครู่

```
5383      gpu      bash kanokpha R 7:07:14 1 cuaaim-dgx-01
5389 interacti 1x1g.5gb peeranon R 0:23 1 cuaaim-dgx-01
(base) [peeranon@cuaaim-frontend1 ~]$ tail -f slurm-5389.out
2023-10-16 10:19:53.670658: I tensorflow/core/common_runtime/gpu/gpu_device.cc:1525] Created device /job:localhost/replica:0/task:0/device:GPU:0
with 3080 MB memory: -> device: 0, name: NVIDIA A100-SXM4-40GB MIG 1g.5gb, pci bus id: 0000:b7:00.0, compute capability: 8.0
x_train shape: (60000, 28, 28, 1)
60000 train samples
10000 test samples
Epoch 1/12
2023-10-16 10:19:57.157052: I tensorflow/stream_executor/cuda/cuda_dnn.cc:368] Loaded cuDNN version 8100
2023-10-16 10:20:03.986238: I tensorflow/stream_executor/cuda/cuda_blas.cc:1786] TensorFlow-32 will be used for the matrix multiplication. This
will only be logged once.
469/469 [=====] - 16s 9ms/step - loss: 27.8481 - accuracy: 0.1473 - val_loss: 4.3643 - val_accuracy: 0.4450
Epoch 2/12
469/469 [=====] - 4s 9ms/step - loss: 11.7221 - accuracy: 0.2547 - val_loss: 1.7504 - val_accuracy: 0.5640
Epoch 3/12
469/469 [=====] - 4s 8ms/step - loss: 5.7511 - accuracy: 0.3115 - val_loss: 1.4240 - val_accuracy: 0.5298
Epoch 4/12
469/469 [=====] - 4s 8ms/step - loss: 3.4242 - accuracy: 0.3176 - val_loss: 1.6281 - val_accuracy: 0.4205
Epoch 5/12
245/469 [=====>.....] - ETA: 1s - loss: 2.6569 - accuracy: 0.3095
```

8. การใช้งาน Rstudio Server ร่วมกับ Slurm

8.1 Login ไปยัง Login node

1) กรณีเชื่อมต่อจากภายนอก ให้ทำการ Login เข้าไปยัง VPN ของมหาวิทยาลัย ให้สามารถเข้าถึง Network วง 161.200.107.0 ได้

2) เข้าใช้งานไปยัง CUAIM Cluster ด้วยการ Secure Shell ผ่านทางเครื่อง cuaim-frontend1 (IP: 161.200.107.77) โดยตัวอย่างในภาพด้านล่าง ใช้โปรแกรม Terminal เชื่อมต่อไปยังระบบ โดยกรอกข้อมูลในช่อง Host Name (or IP address) ในรูปแบบของ <username>@cuaim-frontend1 เช่น peeranon@cuaim-frontend1

```
[root@frontend1 ~]# ssh peeranon@cuaim-frontend1-10g
Password:
Last login: Sat Mar  5 12:26:23 2022 from localhost
[peeranon@cuaim-frontend1 ~]$ sinfo
```

8.2 ตรวจสอบสถานะของ Slurm

1) ใช้คำสั่ง `sinfo` สำหรับตรวจสอบของเครื่องใน Slurm

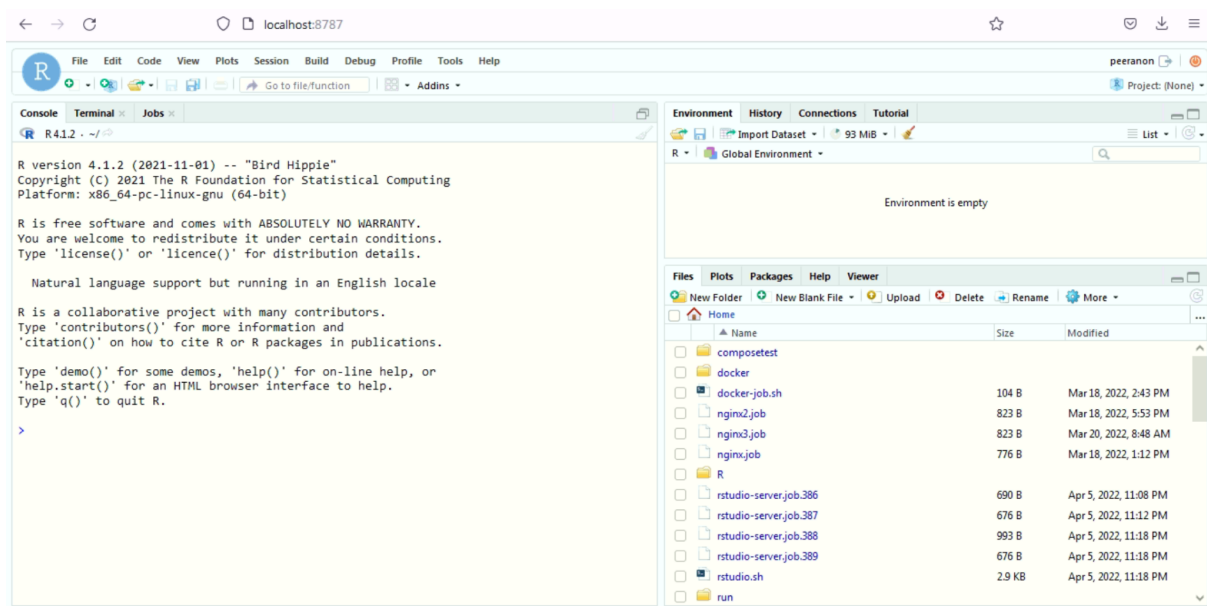
```
[root@cuaim-frontend1 ~]# sinfo
PARTITION      AVAIL  TIMELIMIT  NODES  STATE NODELIST
cpu*           up    infinite     1      mix cuaim-fat-01
gpu            up    infinite     1      idle cuaim-dgx-01
interactive-cpu up    infinite     1      mix cuaim-fat-01
interactive-gpu up    infinite     1      idle cuaim-dgx-01
[root@cuaim-frontend1 ~]#
```

2) ใช้คำสั่ง `scontrol show nodes` เพื่อตรวจสอบ Resource ของระบบ ว่าเพียงพอสำหรับใช้งาน

```
[root@cuaim-frontend1 data]# scontrol show nodes
NodeName=cuaim-dgx-01 Arch=x86_64 CoresPerSocket=64
CPUAlloc=0 CPUtot=128 CPULoad=0.74
AvailableFeatures=(null)
ActiveFeatures=(null)
Gres=gpu:8(S:0-1)
NodeAddr=cuaim-dgx-01 NodeHostName=cuaim-dgx-01 Version=21.08.5
OS=linux 5.4.0-81-generic #91-Ubuntu SMP Thu Jul 15 19:09:17 UTC 2021
RealMemory=1031912 AllocMem=0 FreeMem=1008540 Sockets=2 Boards=1
State=IDLE ThreadsPerCore=1 TmpDisk=0 Weight=1 Owner=N/A MCS_label=N/A
Partitions=gpu
BootTime=2022-03-01T23:20:07 SlurmdStartTime=2022-03-02T10:58:58
LastBusyTime=2022-03-05T12:17:39
CfgTRES=cpu=128,mem=1031912M,billing=128,gres/gpu=8
AllocTRES=
CapWatts=n/a
CurrentWatts=0 AveWatts=0
ExtSensorsJoules=n/s ExtSensorsWatts=0 ExtSensorsTemp=n/s

NodeName=cuaim-fat-01 Arch=x86_64 CoresPerSocket=18
CPUAlloc=0 CPUtot=72 CPULoad=0.01
AvailableFeatures=(null)
ActiveFeatures=(null)
Gres=(null)
NodeAddr=cuaim-fat-01 NodeHostName=cuaim-fat-01 Version=21.08.5
OS=linux 3.10.0-1160.el7.x86_64 #1 SMP Mon Oct 19 16:18:59 UTC 2020
RealMemory=773339 AllocMem=0 FreeMem=680807 Sockets=4 Boards=1
State=IDLE ThreadsPerCore=1 TmpDisk=0 Weight=1 Owner=N/A MCS_label=N/A
Partitions=cpu
BootTime=2022-03-01T16:53:42 SlurmdStartTime=2022-03-01T23:30:51
LastBusyTime=2022-03-05T12:17:17
CfgTRES=cpu=72,mem=773339M,billing=72
AllocTRES=
CapWatts=n/a
CurrentWatts=0 AveWatts=0
ExtSensorsJoules=n/s ExtSensorsWatts=0 ExtSensorsTemp=n/s
```

8.3 การใช้งาน Rstudio Server ผ่าน SLURM



ผู้ใช้งานสามารถรัน Rstudio Server บนเครื่องในระบบ CUAIM และเข้าใช้งานผ่านเครื่องของผู้ใช้งานได้ โดยมี Slurm ทำหน้าที่จัดสรรทรัพยากรให้

การใช้งาน Rstudio Server จะใช้การ Submit Batch Job ผ่าน Slurm เพื่อไปสร้าง Singularity Container ในระบบ

การใช้งาน Rstudio Server

- 1) Start Rstudio Server Batch Job ผ่าน Slurm
- 2) Start SSH Tunnel
- 3) เข้าใช้งานผ่าน Browser บนเครื่องของผู้ใช้งาน

1) Start Rstudio Server Batch Job ผ่าน Slurm

ด้านล่างเป็นตัวอย่างสคริปต์สำหรับ Submit งานของ Rstudio Server เป็น Batch Job ผู้ใช้งานควรปรับค่าต่าง ๆ ของ Slurm (#SBATCH) ตามความเหมาะสม

```
#!/bin/sh
#SBATCH --time=08:00:00
#SBATCH -p cpu
#SBATCH --signal=USR2
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=2
#SBATCH --mem=8192
#SBATCH --output=rstudio-server.job.%j
# customize --output path as appropriate (to a directory readable only by the user!)

# Create temporary directory to be populated with directories to bind-mount in the container
# where writable file systems are necessary. Adjust path as appropriate for your computing environment.
workdir=$(python -c 'import tempfile; print(tempfile.mkdtemp())')
echo $workdir

mkdir -p -m 700 ${workdir}/run ${workdir}/tmp ${workdir}/var/lib/rstudio-server
cat > ${workdir}/database.conf <<END
provider=sqlite
directory=/var/lib/rstudio-server
END

# Set OMP_NUM_THREADS to prevent OpenBLAS (and any other OpenMP-enhanced
# libraries used by R) from spawning more threads than the number of processors
# allocated to the job.
#
# Set R_LIBS_USER to a path specific to rocker/rstudio to avoid conflicts with
# personal libraries from any R installation in the host environment

cat > ${workdir}/rsession.sh <<END
#!/bin/sh
export OMP_NUM_THREADS=${SLURM_JOB_CPUS_PER_NODE}
export R_LIBS_USER=${HOME}/R/rocker-rstudio/4.1
exec rsession "\${@}"
END

chmod +x ${workdir}/rsession.sh

export SINGULARITY_BIND="${workdir}/run:/run,${workdir}/tmp:/tmp,${workdir}/database.conf:/etc/rstudio/database.conf,
${workdir}/rsession.sh:/etc/rstudio/rsession.sh,${workdir}/var/lib/rstudio-server:/var/lib/rstudio-server"

# Do not suspend idle sessions.
# Alternative to setting session-timeout-minutes=0 in /etc/rstudio/rsession.conf
# https://github.com/rstudio/rstudio/blob/v1.4.1106/src/cpp/server/ServerSessionManager.cpp#L126
export SINGULARITYENV_RSTUDIO_SESSION_TIMEOUT=0

export SINGULARITYENV_USER=$(id -un)
export SINGULARITYENV_PASSWORD=$(openssl rand -base64 15)

# get unused socket per https://unix.stackexchange.com/a/132524
# tiny race condition between the python & singularity commands
readonly PORT=$(python -c 'import socket; s=socket.socket(); s.bind(("", 0)); print(s.getsockname()[1]); s.close()')
cat 1>&2 <<END
1. SSH tunnel from your workstation using the following command:

ssh -N -L 8787:${HOSTNAME}:${PORT} ${SINGULARITYENV_USER}@LOGIN-HOST

and point your web browser to http://localhost:8787

2. log in to RStudio Server using the following credentials:

user: ${SINGULARITYENV_USER}
password: ${SINGULARITYENV_PASSWORD}

When done using RStudio Server, terminate the job by:

1. Exit the RStudio Session ("power" button in the top right corner of the RStudio window)
2. Issue the following command on the login node:

sancel -f ${SLURM_JOB_ID}

END

singularity exec --cleanenv /data/container_image/rstudio_4.1.2.sif \
  rserver --server-user ${USER} \
  --www-port ${PORT} \
  --auth-none=0 \
  --auth-pam-helper-path=pam-helper \
  --auth-stay-signed-in-days=30 \
  --auth-timeout-minutes=0 \
  --rsession-path=/etc/rstudio/rsession.sh
printf 'rserver exited' 1>&2
```

โดยไฟล์ rstudio.sh คำอธิบายมีดังนี้

บรรทัดที่	Option	คำอธิบาย
1	#!/bin/sh	Shell Script Template ต้องประกาศเสมอ
2	--time=08:00:00	กำหนดเวลาในการรันงาน (Walltime) ไว้ที่ 8 ชม.
3	-p cpu	Partition (Queue) ที่ต้องการใช้งาน เช่น cpu หรือ gpu - cpu = เครื่อง cuaim-fat-01 - gpu = เครื่อง cuaim-dgx-01
4	--signal=USR2	The --signal=USR2 directive tells SLURM to send SIGUSR2 approximately 60 seconds before the wall time limit is reached. This causes the rsession process to save user's session state to their home directory, so it can be resumed in a subsequent job.
5	--ntasks=1	จำนวน task = 1
6	--cpus-per-task=2	จำนวน CPU core ต่อ task = 2
7	--mem=8192	ขนาด Memory ที่ต้องการ (MB)
8	--output=rstudio-server-%j	ชื่อไฟล์ output ของงาน ต่อท้ายด้วย %j ซึ่งเป็นเลข Job
***	--qos=cpu8	ระบบ QOS ที่ผู้ใช้งานอยู่ เช่น cpu8 เป็นต้น
13	workdir	Working directory จะอยู่ที่ /data/scratch/\${USER}/\${SLURM_JOB_ID}/ เช่น /data/scratch/peeranon/392/
17-20	\${workdir}/database.conf	ประกาศไฟล์สำหรับ Database configuration ใน Rstudio server
29-34	\${workdir}/rsession.sh	ประกาศค่า OMP_NUM_THREADS ตาม CPU Core ที่ได้รับ และ R_LIBS_USER directory สำหรับ R library ของแต่ละ User
38	export SINGULARITY_BIND	ประกาศ path ที่ต้องการ bind ไปยัง Singularity
43	export SINGULARITYENV_RSTUDIO_SESSION_TIMEOUT=0	เป็นการปิดฟังก์ชันการหยุด idle sessions.
45	export SINGULARITYENV_USER	ประกาศชื่อ User สำหรับ login Rstudio
46	export SINGULARITYENV_PASSWORD	ประกาศชื่อ Password แบบเข้ารหัส สำหรับ login Rstudio
49	readonly PORT	อ่านค่า PORT ที่วางอยู่ในระบบ เพื่อให้ Singularity ทำการ map

50-59	cat 1>&2 “...”	อธิบายวิธีการใช้งาน Rstudio
71-78	singularity exec /data/container_image/rstudio_4.1.2.sif	คำสั่งสำหรับรัน Rstudio server บน Singularity

เมื่อแก้ไขไฟล์สคริปต์เสร็จแล้ว ให้ Submit งานด้วยคำสั่ง **sbatch**

```
[peeranon@cuaim-frontendl ~]$ sbatch rstudio.sh
Submitted batch job 392
```

2) Start SSH Tunnel

เมื่อ Submit งานเรียบร้อยแล้ว Rstudio Server จะเริ่มทำงานและพร้อมแก่การเชื่อมต่อ โดยผู้ใช้งานสามารถรันคำสั่ง `squeue` เพื่อตรวจสอบงานที่ผู้ใช้งานรันอยู่ สถานะที่ขึ้นว่า R ในคอลัมน์ ST บ่งบอกว่างานกำลังรันอยู่ ถ้าขึ้นสถานะ PD ให้รอสักครู่ แล้วจึงจะสามารถใช้งานได้ ไฟล์ Log ที่จะอธิบายข้อมูลการใช้งาน จากตัวอย่าง จะเป็นชื่อ `rstudio-server.job.%j` โดย `%j` เป็นเลข Job ที่รัน เช่น 392

```
[peeranon@cuaim-frontendl ~]$ squeue
  JOBID PARTITION   NAME   USER  ST       TIME  NODES NODELIST(REASON)
    116        cpu peeranon peeranon  R 18-23:11:03      1 cuaim-fat-01
    392        cpu rstudio. peeranon  R      0:02      1 cuaim-fat-01
```

ตัวอย่างเนื้อหาในไฟล์ rstudio-server.job.392 จะมีวิธีการใช้งานดังภาพด้านล่าง โดยให้ตัวอักษรในกรอบสีแดง ใช้สำหรับการทำ SSH Tunnel และ URL ในกรอบสีเขียวเป็น URL สำหรับใช้งานบน Web Browser

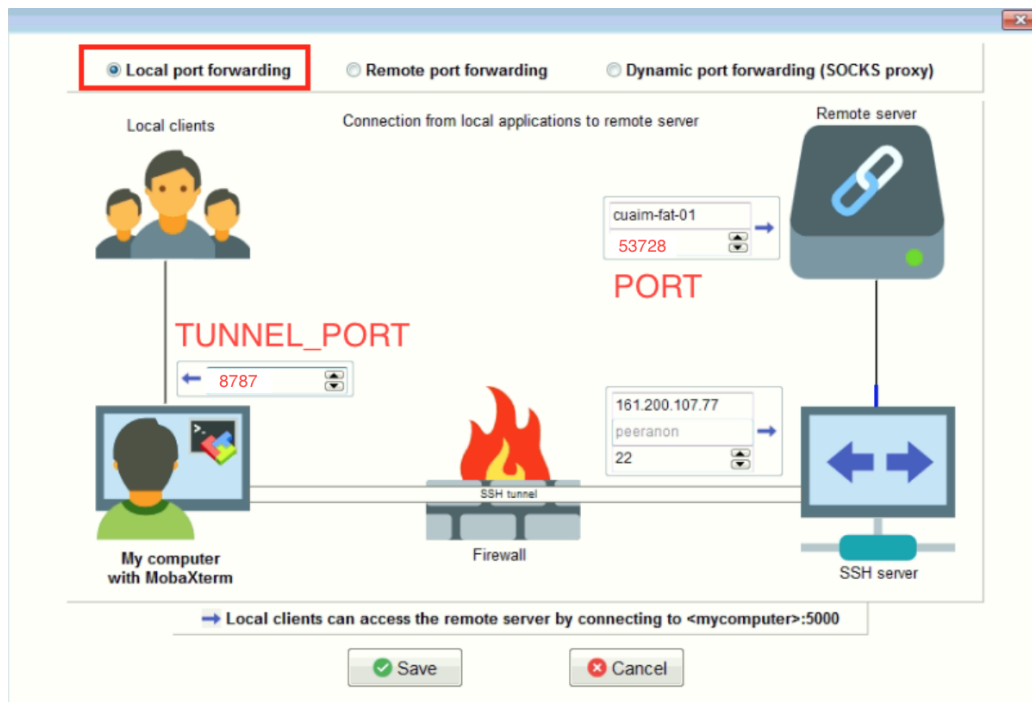
```
[peeranon@cuaaim-frontend1 ~]$ cat rstudio-server.job.392
=====
SLURM_JOB_ID = 392
SLURM_JOB_NAME = rstudio.sh
SLURM_NODELIST = cuaaim-fat-01
=====
/data/scratch/peeranon/392/tmpBcMdQo
1. SSH tunnel from your workstation using the following command:
ssh -N -L 8787:cuaaim-fat-01:53728 peeranon@LOGIN-HOST
and point your web browser to http://localhost:8787
2. log in to RStudio Server using the following credentials:
user: peeranon
password: N08MxAoUIRHsGLiljgTu
When done using RStudio Server, terminate the job by:
1. Exit the RStudio Session ("power" button in the top right corner of the RStudio window)
2. Issue the following command on the login node:
scancel -f 392
```

สำหรับผู้ใช้ Mac OS และ Linux ให้เปิด Terminal แล้วรันคำสั่งตามกรอบสีแดง โดยอาจต้องเปลี่ยนคำว่า LOGIN-HOST เป็น IP Address = 161.200.107.77 แล้วกรอก password ของผู้ใช้งานลงไป ตามภาพด้านล่าง

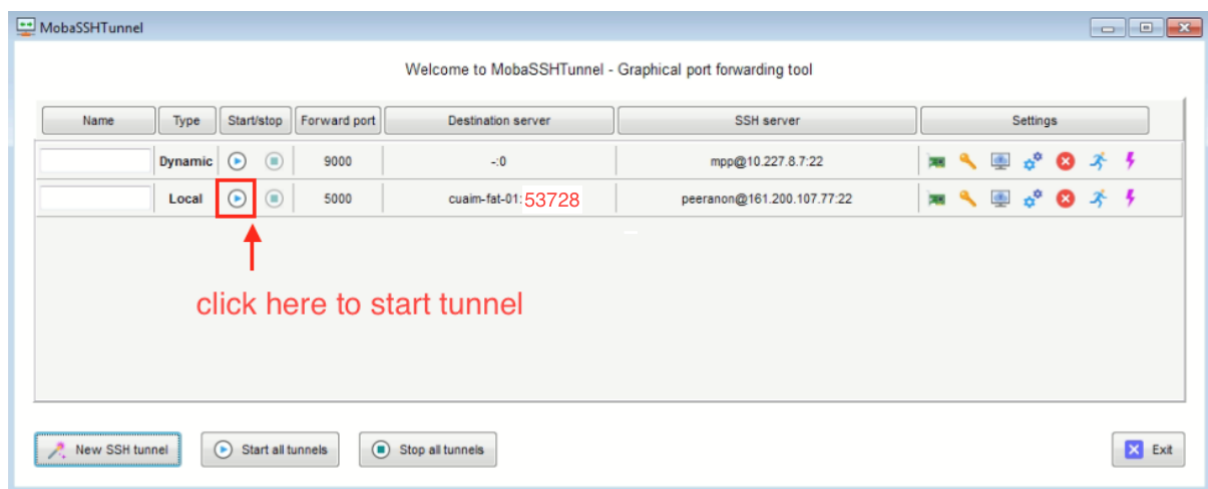
```
Peeranons-MacBook-Pro:~ peeranon$ ssh -N -L 8787:cuaaim-fat-01:53728 peeranon@161.200.107.77
```

การใช้งานบน Windows ให้ใช้โปรแกรม MobaXterm โดยตรวจสอบข้อมูลเพิ่มเติมเกี่ยวกับการทำ SSH Tunnel ได้ โดยทำดังนี้

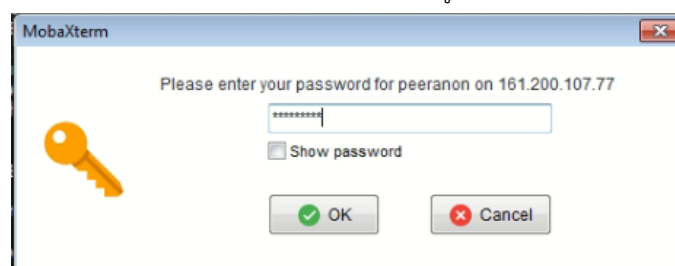
1. ภายในแถบ Tools เลือก "MobaSSHTunnel (port forwarding)"
2. คลิกที่ปุ่ม "New SSH Tunnel"
3. เลือกไปที่ปุ่ม "Local port forwarding"



4. กรอกข้อมูลที่แสดงมาจาก output ของ Batch job ตามภาพ
5. กด Save.



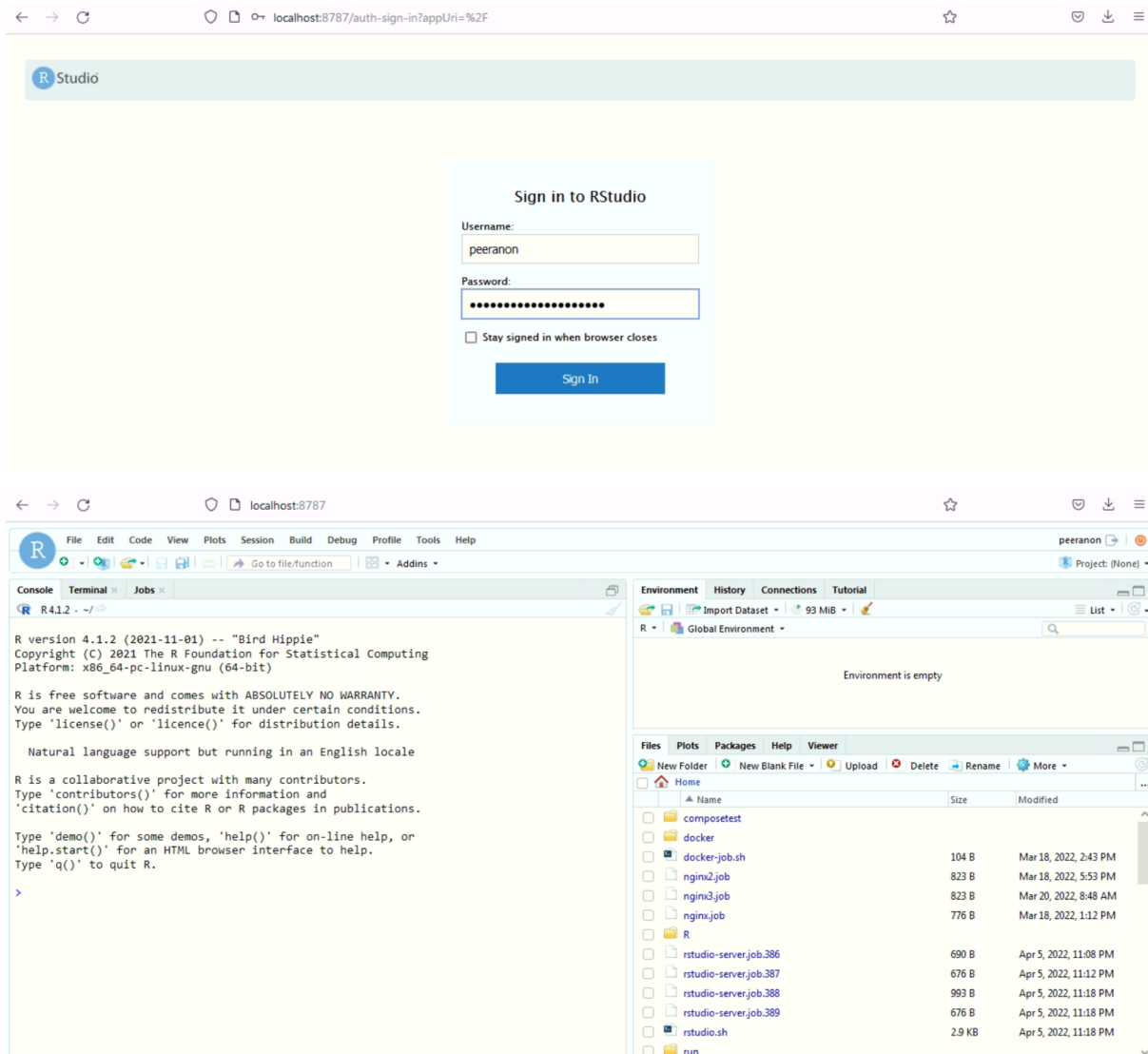
6. คลิกที่ปุ่ม play ได้คอลัมน์ Start/Stop
7. ถ้าการเชื่อมต่อสำเร็จ ระบบจะให้กรอกรหัสผ่านของผู้ใช้งาน



8. เมื่อเข้าใช้งานเสร็จเรียบร้อย ให้ลองเข้าหน้า Rstudio บน Browser

3) เข้าใช้งานผ่าน Browser บนเครื่องของผู้ใช้งาน

หลังจากที่เปิดใช้งาน SSH Tunnel เสร็จเรียบร้อยแล้ว ให้เปิด Browser ของผู้ใช้งาน จากนั้นไปที่ URL = <http://localhost:8787> จะปรากฏหน้าเว็บของ Rstudio Server



การเพิ่ม Package Dependencies สำหรับ Library บน Rstudio

ในการทำงานของผู้ใช้งานแต่ละคน ก็จะมีลักษณะการใช้งานใน Environment รวมถึง Library ที่แตกต่างกันออกไป ในการติดตั้ง Library เพิ่มเติมบางตัวนั้น อาจจำเป็นต้องมี Package Dependencies ที่แตกต่างกัน

สิ่งที่ผู้พัฒนาโปรแกรม R ควรทราบ คือ Library ที่ตนเองต้องการใช้นั้น จะต้องมี Software Prerequisite ไต่บ้าง ยกตัวอย่างเช่น Library ชื่อ **Velocyto.R** ที่จะต้องมี Dependencies บน System library ที่ต้องติดตั้ง **apt-get install libboost-dev** ดังภาพด้านล่าง ในกรณีที่ต้องการ System library เพิ่มเติมเช่นนี้ ให้ติดต่อผู้ดูแลระบบเพื่อทำการติดตั้ง

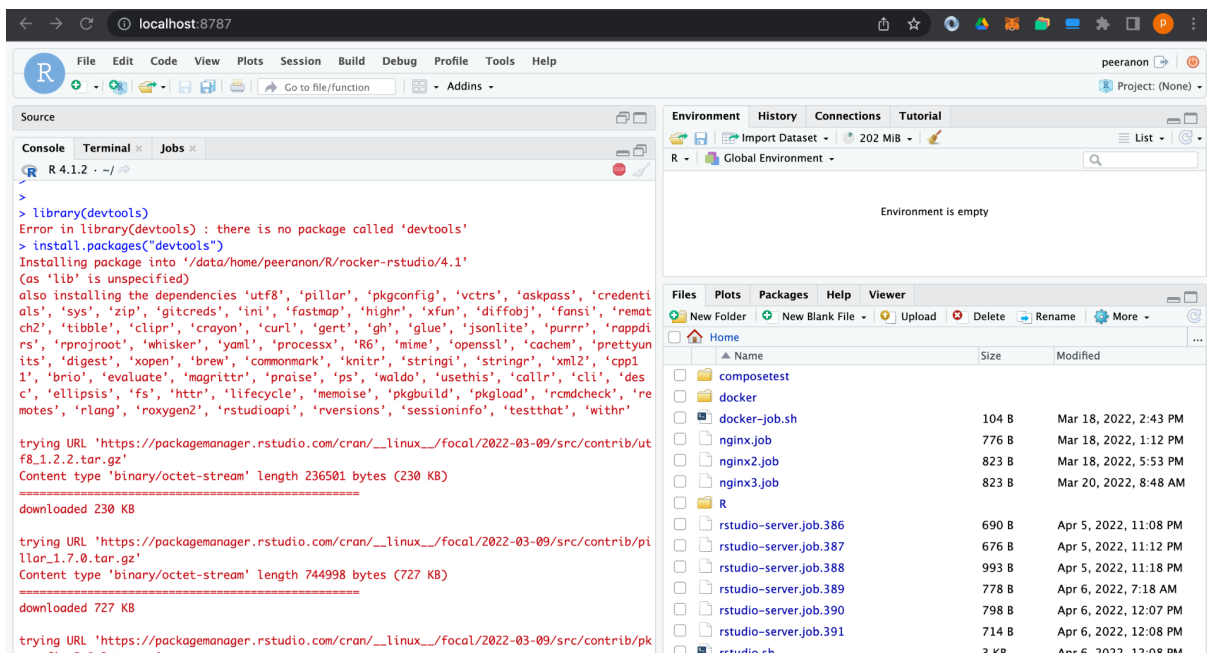
Installation

The easiest way to install velocyto.R is using devtools::install_github() from R:

```
library(devtools)
install_github("velocyto-team/velocyto.R")
```

You need to have boost (e.g. `sudo apt-get install libboost-dev`) and openmp libraries installed. You can see detailed installation commands in the `dockers/debian9/Dockerfile`.

ตัวอย่างการติดตั้ง library เพิ่มเติมบน Rstudio server



การหยุดใช้งาน Rstudio server

ในการหยุดใช้งาน Rstudio server

1. กดปุ่ม Power ที่มุมขวาบนของหน้าต่าง RStudio
2. สั่งยกเลิกงานดังกล่าวบน Slurm ด้วยคำสั่ง `scancel -f [jobid]` เช่น `scancel -f 392` ดังกรอบสีเหลืองในภาพด้านล่าง

```
[peeranon@cuaim-frontend1 ~]$ cat rstudio-server.job.392
=====
SLURM_JOB_ID = 392
SLURM_JOB_NAME = rstudio.sh
SLURM_NODELIST = cuaim-fat-01
=====
/data/scratch/peeranon/392/tmpBcMdQo
1. SSH tunnel from your workstation using the following command:

  ssh -N -L 8787:cuaim-fat-01:53728 peeranon@LOGIN-HOST

  and point your web browser to http://localhost:8787

2. log in to RStudio Server using the following credentials:

  user: peeranon
  password: N08MxAoUIRHsGLiljgTu

When done using RStudio Server, terminate the job by:

1. Exit the RStudio Session ("power" button in the top right corner of the RStudio window)
2. Issue the following command on the login node:

  scancel -f 392
```

9. การใช้งาน Jupyter notebook บน TensorFlow Singularity image ร่วมกับ Slurm

9.1 Login ไปยัง Login node

1) กรณีเชื่อมต่อจากภายนอก ให้ทำการ Login เข้าไปยัง VPN ของมหาวิทยาลัย ให้สามารถเข้าถึง Network วง 161.200.107.0 ได้

2) เข้าใช้งานไปยัง CUAIM Cluster ด้วยการ Secure Shell ผ่านทางเครื่อง cuaim-frontend1 (IP: 161.200.107.77) โดยตัวอย่างในภาพด้านล่าง ใช้โปรแกรม Terminal เชื่อมต่อไปยังระบบ โดยกรอกข้อมูลในช่อง Host Name (or IP address) ในรูปแบบของ <username>@cuaim-frontend1 เช่น peeranon@cuaim-frontend1

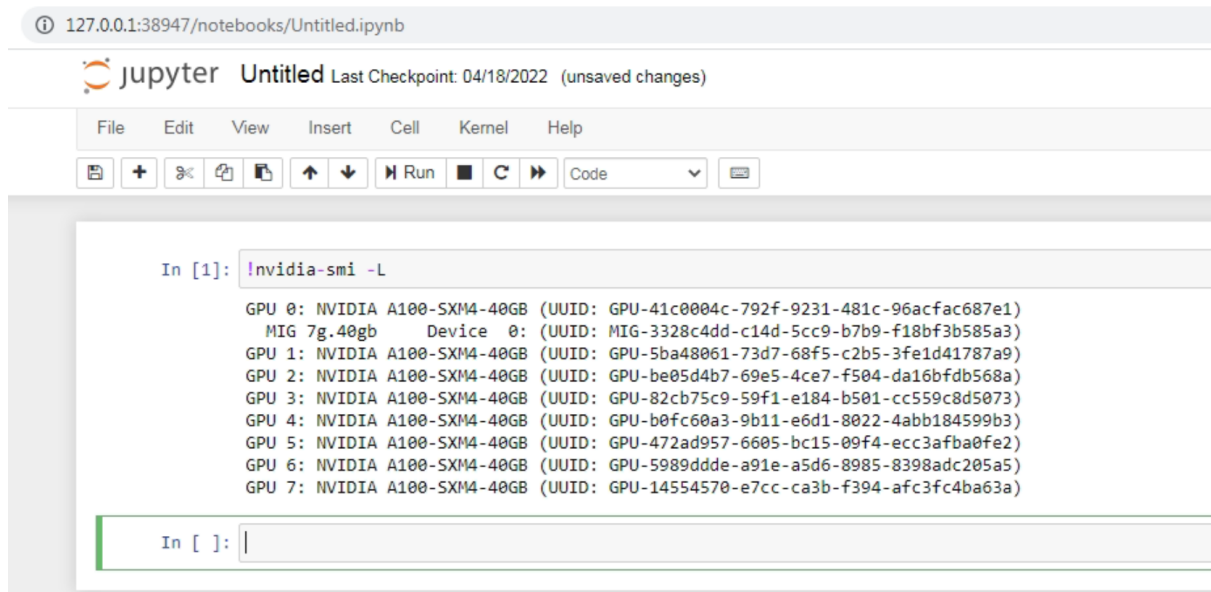
```
[root@frontend1 ~]# ssh peeranon@cuaim-frontend1-10g
Password:
Last login: Sat Mar  5 12:26:23 2022 from localhost
[peeranon@cuaim-frontend1 ~]$ sinfo
```

9.2 ตรวจสอบสถานะของ Slurm

1) ใช้คำสั่ง `sinfo` สำหรับตรวจสอบของเครื่องใน Slurm

```
[root@cuaim-frontend1 ~]# sinfo
PARTITION      AVAIL  TIMELIMIT  NODES  STATE NODELIST
cpu*            up    infinite     1    mix  cuaim-fat-01
gpu             up    infinite     1    idle cuaim-dgx-01
interactive-cpu up    infinite     1    mix  cuaim-fat-01
interactive-gpu up    infinite     1    idle cuaim-dgx-01
[root@cuaim-frontend1 ~]#
```

9.3 การใช้งาน Jupyter notebook บน TensorFlow Singularity image ร่วมกับ Slurm



```
In [1]: !nvidia-smi -L

GPU 0: NVIDIA A100-SXM4-40GB (UUID: GPU-41c0004c-792f-9231-481c-96acfac687e1)
MIG 7g.40gb Device 0: (UUID: MIG-3328c4dd-c14d-5cc9-b7b9-f18bf3b585a3)
GPU 1: NVIDIA A100-SXM4-40GB (UUID: GPU-5ba48061-73d7-68f5-c2b5-3fe1d41787a9)
GPU 2: NVIDIA A100-SXM4-40GB (UUID: GPU-be05d4b7-69e5-4ce7-f504-da16bfd568a)
GPU 3: NVIDIA A100-SXM4-40GB (UUID: GPU-82cb75c9-59f1-e184-b501-cc559c8d5073)
GPU 4: NVIDIA A100-SXM4-40GB (UUID: GPU-b0fc60a3-9b11-e6d1-8022-4abb184599b3)
GPU 5: NVIDIA A100-SXM4-40GB (UUID: GPU-472ad957-6605-bc15-09f4-ecc3afba0fe2)
GPU 6: NVIDIA A100-SXM4-40GB (UUID: GPU-5989ddde-a91e-a5d6-8985-8398adc205a5)
GPU 7: NVIDIA A100-SXM4-40GB (UUID: GPU-14554570-e7cc-ca3b-f394-afc3fc4ba63a)
```

ผู้ใช้งานสามารถรัน Jupyter notebook บน TensorFlow container image บนเครื่องในระบบ CUAIM และเข้าใช้งานผ่านเครื่องของผู้ใช้งานได้ โดยมี Slurm ทำหน้าที่จัดสรรทรัพยากรให้

การใช้งาน Jupyter notebook บน TensorFlow จะใช้การ Submit Batch Job ผ่าน Slurm เพื่อไปสร้าง Singularity Container ในระบบ

การใช้งาน Jupyter notebook บน TensorFlow

- 1) Start Jupyter notebook Batch Job ผ่าน Slurm
- 2) Start SSH Tunnel
- 3) เข้าใช้งานผ่าน Browser บนเครื่องของผู้ใช้งาน

1) Start Jupyter notebook Batch Job ผ่าน Slurm

ด้านล่างเป็นตัวอย่างสคริปต์สำหรับ Submit งานของJupyter notebook เป็น Batch Job ผู้ใช้งานควรปรับค่าต่าง ๆ ของ Slurm (#SBATCH) ตามความเหมาะสม

```
[peeranon@cuaim-frontend1 ~]$ cat scripts/pytorch-jupyter.sh
1 [peeranon@cuaim-frontend1 ~]$ cat scripts/pytorch-jupyter.sh
2 #!/bin/sh
3 #SBATCH --time=08:00:00
4 #SBATCH -p gpu
5 #SBATCH --signal=USR2
6 #SBATCH --ntasks=1
7 #SBATCH --cpus-per-task=2
8 #SBATCH --mem=8192
9 #SBATCH --gres=gpu:7g.40gb:1
10 #SBATCH --qos=gpu40g
11 #SBATCH --output=pytorch-jupyter.job.%j
12
13 # Create temporary directory to be populated with directories to bind-mount in the container
14 # where writable file systems are necessary. Adjust path as appropriate for your computing environment.
15 workdir=$(python -c 'import tempfile; print(tempfile.mkdtemp())')
16 echo ${workdir}
17
18 export SINGULARITY_BIND="${workdir},${PWD}"
19
20 XDG_RUNTIME_DIR=""
21 node=$(hostname -s)
22 user=$(whoami)
23 cluster=$(hostname)
24
25 readonly PORT=$(python -c 'import socket; s=socket.socket(); s.bind(("", 0)); print(s.getsockname()[1]); s.close()')
26
27 cat 1>&2 <<END
28 1. SSH tunnel from your workstation using the following command:
29 - MacOS or linux terminal
30   ssh -N -L ${PORT}:${HOSTNAME}:${PORT} $user@161.200.107.77
31 - Windows MobaXterm info
32   Forwarded port: same as remote port
33   Remote server: ${node}
34   Remote port: ${PORT}
35   SSH server: 161.200.107.77
36   SSH login: $user
37   SSH port: 22
38
39 2. cat the output file to get URL at the very bottom of the file and copy it.
40   "http://127.1.0.1:${PORT}/?token=..."
41
42 3. After SSH Tunnel done, Place the URL in with token to the browser
43 When done using Jupyter Notebook, terminate the job by:
44
45 1. Exit the Jupyter Notebook Session
46 2. Issue the following command on the login node:
47
48   scancel -f ${SLURM_JOB_ID}
49
50 END
51
52 singularity exec /data/container_image/pytorch-22.03-py3.sif \
53   jupyter-notebook --no-browser --port=${PORT} --ip=0.0.0.0
54   printf 'Jupyter notebook exited! 1-52'
Line 17, Column 1
```

โดยไฟล์ rstudio.sh คำอธิบายมีดังนี้

บรรทัดที่	Option	คำอธิบาย
1	#!/bin/sh	Shell Script Template ต้องประกาศเสมอ
2	--time=08:00:00	กำหนดเวลาในการทำงาน (Walltime) ไว้ที่ 8 ชม.
3	-p cpu	Partition (Queue) ที่ต้องการใช้งาน เช่น cpu หรือ gpu - cpu = เครื่อง cuaim-fat-01

		- gpu = เครื่อง cuaim-dgx-01
4	--signal=USR2	The --signal=USR2 directive tells SLURM to send SIGUSR2 approximately 60 seconds before the wall time limit is reached. This causes the rsession process to save user's session state to their home directory, so it can be resumed in a subsequent job.
5	--ntasks=1	จำนวน task = 1
6	--cpus-per-task=2	จำนวน CPU core ต่อ task = 2
7	--mem=8192	ขนาด Memory ที่ต้องการ (MB)
8	--output=tensorflow-jupyter-%j	ชื่อไฟล์ output ของงาน ต่อท้ายด้วย %j ซึ่งเป็นเลข Job
9	--gres=gpu:7g.40gb:1	จำนวน GPU ที่ต้องการ
***	--qos=gpu40g	ระบบ QOS ที่ผู้ใช้งานอยู่ เช่น gpu40g เป็นต้น
16	workdir	Working directory จะอยู่ที่ /data/scratch/\${USER}/\${SLURM_JOB_ID}/ เช่น /data/scratch/peeranon/392/
18	export SINGULARITY_BIND	ประกาศ path ที่ต้องการ bind ไปยัง Singularity
25	readonly PORT	อ่านค่า PORT ที่วางอยู่ในระบบ เพื่อให้ Singularity ทำการ map
27-50	cat 1>&2 “...”	อธิบายวิธีการเข้าใช้งาน Jupyter
52-53	singularity exec --nv /data/container_image/pytorch.sif	คำสั่งสำหรับรัน Jupyter notebook บน Singularity

เมื่อแก้ไขไฟล์สคริปต์เสร็จแล้ว ให้ Submit งานด้วยคำสั่ง **sbatch**

```
[peeranon@cuaim-frontend1 ~]$ sbatch scripts/tensorflow-jupyter.sh
Submitted batch job 877
```

2) Start SSH Tunnel

เมื่อ Submit งานเรียบร้อยแล้ว Jupyter Notebook จะเริ่มทำงานและพร้อมแก่การเชื่อมต่อ โดยผู้ใช้งานสามารถรันคำสั่ง `squeue` เพื่อตรวจสอบงานที่ผู้ใช้งานรันอยู่ สถานะที่ขึ้นว่า R ในคอลัมน์ ST บ่งบอกว่างานกำลังรันอยู่ ถ้าขึ้นสถานะ PD ให้รอสักครู่ แล้วจึงจะสามารถเข้าใช้งานได้ ไฟล์ Log ที่จะอธิบายข้อมูลการเข้าใช้งาน จากตัวอย่าง จะเป็นชื่อ `tensorflow.job.%j` โดย `%j` เป็นเลข Job ที่รัน เช่น 878

```
[peeranon@cuaim-frontend1 ~]$ squeue
      JOBID PARTITION    NAME   USER  ST       TIME  NODES NODELIST(REASON)
      636      cpu    webapp   kantam  R   105-11:06:48      1 cuaim-fat-01
      868      cpu      cgen     cgen   R    1-12:55:02      1 cuaim-fat-01
      836      gpu    sl/V tpakornc  R   18-03:12:23      1 cuaim-dgx-01
      837      gpu    s3/V tpakornc  R   18-03:12:06      1 cuaim-dgx-01
      878      gpu   1x40GB peeranon  R         0:02      1 cuaim-dgx-01
```

ตัวอย่างเนื้อหาในไฟล์ tensorflow-jupyter.job.878 จะมีวิธีการเข้าใช้งานดังภาพด้านล่าง โดยให้ตัวอักษรในกรอบสีแดงใช้สำหรับการทำ SSH Tunnel และ URL ในกรอบสีเขียวเป็น URL สำหรับใช้งานบน Web Browser

```
SLURM_NODELIST = cuaim-dgx-01
=====
/data/scratch/peeranon/877/tmplwbk0V
1. SSH tunnel from your workstation using the following command:

- MacOS or linux terminal
  ssh -N -L 38947:cuaim-dgx-01:38947 peeranon@161.200.107.77

- Windows MobaXterm info
  Forwarded port: same as remote port
  Remote server: cuaim-dgx-01
  Remote port: 38947
  SSH server: 161.200.107.77
  SSH login: peeranon
  SSH port: 22

2. cat the output file to get URL at the very bottom of the file and copy it.

"http://127.1.0.1:38947/?token=..."

3. After SSH Tunnel done, Place the URL in with token to the browser

When done using Jupyter Notebook, terminate the job by:

1. Exit the Jupyter Notebook Session
2. Issue the following command on the login node:

  scancel -f 877

2022-09-07 20:48:50.738905: I tensorflow/stream_executor/platform/default/dso_loader.cc:49] Successfully opened dynamic library libcudart.so.11.0
[I 20:48:52.275 NotebookApp] jupyter_tensorboard extension loaded.
[I 20:48:52.389 NotebookApp] JupyterLab extension loaded from /usr/local/lib/python3.8/dist-packages/jupyterlab
[I 20:48:52.389 NotebookApp] JupyterLab application directory is /usr/local/share/jupyter/lab
[I 20:48:52.391 NotebookApp] [JupyterText Server Extension] NotebookApp.contents_manager_class is (a subclass of) jupyter_text.TextFileContentsManager already - OK
[I 20:48:52.391 NotebookApp] Serving notebooks from local directory: /data/home/peeranon
[I 20:48:52.391 NotebookApp] The Jupyter Notebook is running at:
[I 20:48:52.391 NotebookApp] http://hostname:8888/?token=e6e951f88d6bbbfbaba8388548fbf5a7dc0aa566e4cf1df4a
[I 20:48:52.391 NotebookApp] or http://127.0.0.1:38947/?token=e6e951f88d6bbbfbaba8388548fbf5a7dc0aa566e4cf1df4a
[I 20:48:52.391 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 20:48:52.395 NotebookApp]

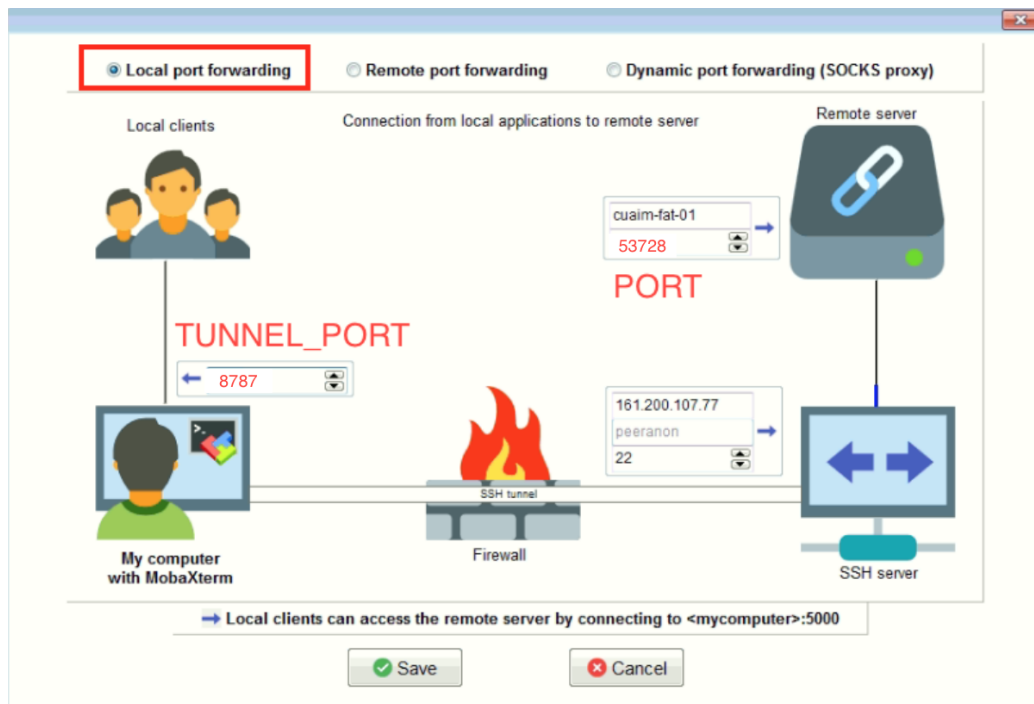
To access the notebook, open this file in a browser:
  file:///data/home/peeranon/.local/share/jupyter/runtime/nbserver-2720672-open.html
Or copy and paste one of these URLs:
  http://hostname:8888/?token=e6e951f88d6bbbfbaba8388548fbf5a7dc0aa566e4cf1df4a
  or http://127.0.0.1:38947/?token=e6e951f88d6bbbfbaba8388548fbf5a7dc0aa566e4cf1df4a
```

สำหรับผู้ Mac OS และ Linux ให้เปิด Terminal แล้วรันคำสั่งตามกรอบสีแดง โดยอาจต้องเปลี่ยนคำว่า LOGIN-HOST เป็น IP Address = 161.200.107.77 แล้วกรอก password ของผู้ใช้งานลงไป ตามภาพด้านล่าง

```
[user.DELL_7010] > ssh -N -L 57583:cuaim-dgx-01:57583 peeranon@161.200.107.77
Warning: Permanently added '161.200.107.77' (RSA) to the list of known hosts.
```

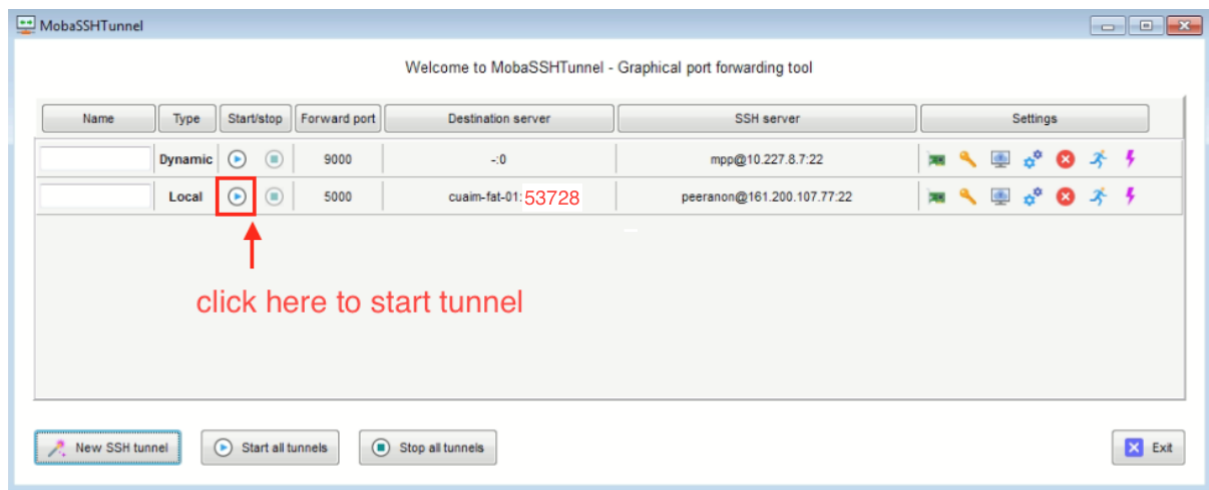
การใช้งานบน Windows ให้ใช้โปรแกรม MobaXterm โดยตรวจสอบข้อมูลเพิ่มเติมเกี่ยวกับการทำ SSH Tunnel ได้ โดยทำดังนี้

9. ภายในแถบ Tools เลือก "MobaSSHTunnel (port forwarding)"
10. คลิกที่ปุ่ม "New SSH Tunnel"
11. เลือกไปที่ปุ่ม "Local port forwarding"



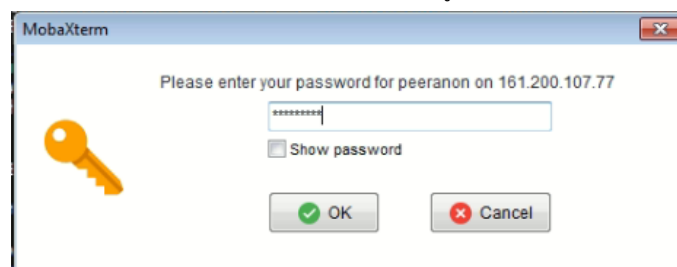
12. กรอกรหัสที่แสดงมาจาก output ของ Batch job ตามภาพ

13. กด Save.



14. คลิกที่ปุ่ม play ได้คอลัมน์ Start/Stop

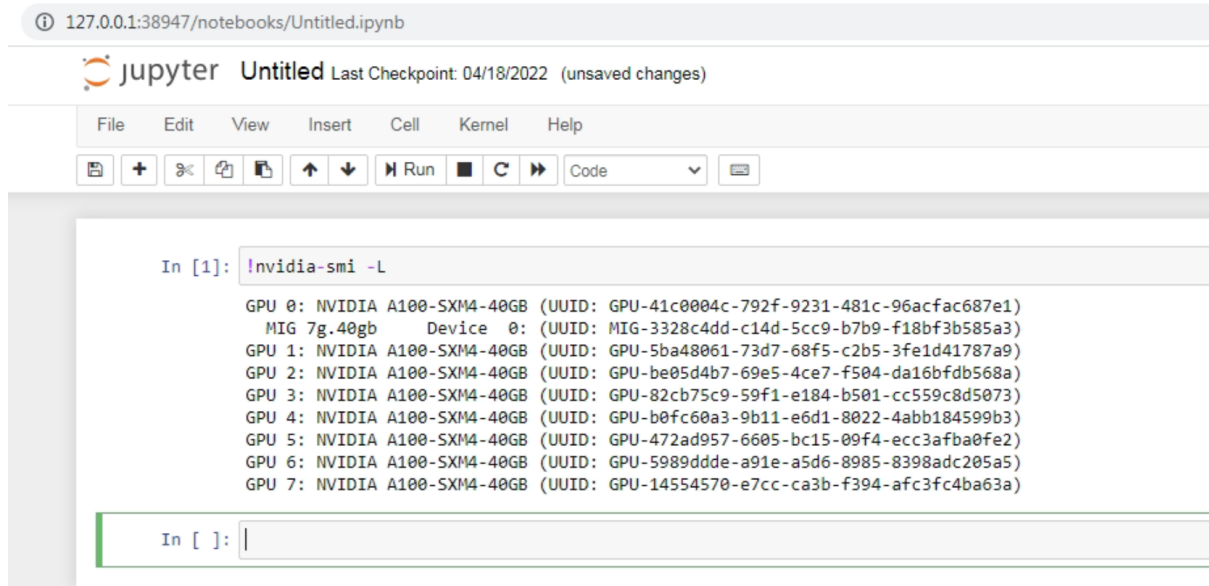
15. ถ้าการเชื่อมต่อสำเร็จ ระบบจะให้กรอกรหัสผ่านของผู้ใช้งาน



16. เมื่อเข้าใช้งานเสร็จเรียบร้อย ให้ลองเข้าหน้า Rstudio บน Browser

3) เข้าใช้งานผ่าน Browser บนเครื่องของผู้ใช้งาน

หลังจากที่เปิดใช้งาน SSH Tunnel เสร็จเรียบร้อยแล้ว ให้เปิด Browser ของผู้ใช้งาน จากนั้นไปที่ URL = <http://127.0.0.1:38947/?token=e6e951f88d6bbb> จะปรากฏหน้าเว็บของ Jupyter Notebook



```
In [1]: !nvidia-smi -L

GPU 0: NVIDIA A100-SXM4-40GB (UUID: GPU-41c0004c-792f-9231-481c-96acf687e1)
        MIG 7g.40gb      Device 0: (UUID: MIG-3328c4dd-c14d-5cc9-b7b9-f18bf3b585a3)
GPU 1: NVIDIA A100-SXM4-40GB (UUID: GPU-5ba48061-73d7-68f5-c2b5-3fe1d41787a9)
GPU 2: NVIDIA A100-SXM4-40GB (UUID: GPU-be05d4b7-69e5-4ce7-f504-da16bfbdb568a)
GPU 3: NVIDIA A100-SXM4-40GB (UUID: GPU-82cb75c9-59f1-e184-b501-cc559c8d5073)
GPU 4: NVIDIA A100-SXM4-40GB (UUID: GPU-b0fc60a3-9b11-e6d1-8022-4abb184599b3)
GPU 5: NVIDIA A100-SXM4-40GB (UUID: GPU-472ad957-6605-bc15-09f4-ecc3afbaf0fe2)
GPU 6: NVIDIA A100-SXM4-40GB (UUID: GPU-5989ddde-a91e-a5d6-8985-8398adc205a5)
GPU 7: NVIDIA A100-SXM4-40GB (UUID: GPU-14554570-e7cc-ca3b-f394-afc3fc4ba63a)

In [ ]: |
```

การหยุดใช้งาน Jupyter notebook บน TensorFlow Singularity Image

ในการหยุดใช้งาน Jupyter notebook

3. กดปุ่ม File -> Exit บนของหน้าต่าง Jupyter notebook
4. สั่งยกเลิกงานดังกล่าวบน Slurm ด้วยคำสั่ง `scancel -f [jobid]` เช่น `scancel -f 392` ดังกรอบสีเหลืองในภาพด้านล่าง

3. After SSH Tunnel done, Place the URL in with token to the browser

When done using Jupyter Notebook, terminate the job by:

1. Exit the Jupyter Notebook Session
2. Issue the following command on the login node:

```
scancel -f 877
```

